

**BỘ NÔNG NGHIỆP VÀ MÔI TRƯỜNG
PHÂN HIỆU TRƯỜNG ĐẠI HỌC THỦY LỢI
BỘ MÔN CÔNG NGHỆ THÔNG TIN**



BÁO CÁO MÔN HỌC MÁY

Tên đề tài:

**PHÂN TÍCH, XỬ LÝ DỮ LIỆU DỊCH BỆNH
TRÊN GÀ THÔNG QUA HÌNH ẢNH VÀ TIẾN HÀNH DỰ ĐOÁN**

Giảng viên hướng dẫn:	Vũ Thị Hạnh
Sinh viên thực hiện:	<i>Nguyễn Quốc Bảo Cao Thị Ngọc Luyện Lục Duy Quang</i>
Mssv:	<i>2351067085 2351067099 2351067093</i>
Lớp:	<i>S26-65CNTT</i>

Lời Cảm Ơn

Để hoàn thành được bài tiểu luận này, em xin chân thành cảm ơn Ban Giám hiệu, các khoa, phòng và quý thầy, cô của trường Phân hiệu Đại học Thủy Lợi, những người đã tận tình giúp đỡ và tạo điều kiện cho em trong quá trình học tập. Đặc biệt, em xin gửi lời cảm ơn sâu sắc đến cô Vũ Thị Hạnh - người đã trực tiếp giảng dạy và hướng dẫn em thực hiện bài tiểu luận này bằng tất cả lòng nhiệt tình và sự quan tâm sâu sắc.

Trong quá trình thực hiện bài tiểu luận này, do hiểu biết còn nhiều hạn chế nên bài làm khó tránh khỏi những thiếu sót. Em rất mong nhận được những lời góp ý của quý thầy cô để bài tiểu luận ngày càng hoàn thiện hơn.

Em xin chân thành cảm ơn!

MỤC LỤC

Chương 1: Giới thiệu bài toán	5
1. Giới thiệu chung:	5
2. Mục tiêu nghiên cứu:	5
3. Phương pháp nghiên cứu:	5
Chương 2: Mô tả dữ liệu và tiền xử lý dữ liệu:	6
1. Mô tả dữ liệu:	6
Chương 3: Phương pháp và mô hình học máy áp dụng	7
1. Mô hình ResNet50:	7
1.1 Giới thiệu chung:	7
1.2 Source Code:	9
2. Mô hình EfficientNetB0:	28
2.1 Giới thiệu chung	28
2.2 Source code	30
3. Mô hình CNN	42
Chương 4: Đánh giá mô hình.	47
1. EfficientNetB0.	47
1.1 Biểu đồ Accuracy.	47
1.2 Biểu đồ Loss.	48
1.3 Ma trận nhầm lẫn(confusion matrix).	49
1.4 Các chỉ số.	50
2. Resnet50:	52
3. CNN	59
Chương 5: Kết quả bước đầu và nhận xét.....	60
1. Ưu và nhược điểm:	60
2. Kết luận:	61
Chương 6: WEB chẩn đoán	61
1. Giới thiệu chung:	61
2. Source code:	61
Chương 7: Các tài liệu tham khảo	70

This image shows a full page of white paper with horizontal dashed lines, typical of primary-ruled notebook paper. The lines are evenly spaced and run across the entire width of the page. There are no margins, text, or other markings present.

Ký và ghi rõ họ tên

4

Chương 1: Giới thiệu bài toán

1. Giới thiệu chung:

Trong lĩnh vực nông nghiệp và chăn nuôi, việc phát hiện sớm các loại bệnh ở gia cầm đóng vai trò quan trọng trong việc:

- ✓ Giảm tỷ lệ chết
- ✓ Hạn chế lây lan dịch bệnh
- ✓ Giảm thiểu thiệt hại kinh tế cho người chăn nuôi

Tuy nhiên, việc chẩn đoán bệnh thường đòi hỏi kiến thức chuyên môn và kinh nghiệm của bác sĩ thú y. Với sự phát triển của Học máy (Machine Learning) và đặc biệt là Học sâu (Deep Learning) trong xử lý ảnh, việc tự động nhận diện bệnh thông qua hình ảnh trở nên khả thi và hiệu quả hơn.

2. Mục tiêu nghiên cứu:

Trong đề án này, nhóm chúng tôi tập trung giải quyết bài toán phân loại bệnh ở gà dựa trên hình ảnh, sử dụng tập dữ liệu Chicken Disease Dataset từ Kaggle. Mục tiêu của bài toán là:

Với một ảnh đầu vào của gà, mô hình sẽ dự đoán gà đó mắc bệnh gì hoặc khỏe mạnh.

Nhóm tiến hành thử nghiệm và so sánh hiệu quả của các mô hình học sâu gồm CNN, EfficientNetB0 và ResNet50.

3. Phương pháp nghiên cứu:

Nghiên cứu lý thuyết: Tìm hiểu các tài liệu, sách, bài viết và nguồn tài nguyên trực tuyến liên quan đến xử lý dữ liệu ảnh, machine learning, deep learning.

Nghiên cứu thực nghiệm: Nhóm tiến hành thử nghiệm và so sánh hiệu quả của các mô hình học sâu gồm CNN, EfficientNetB0 và ResNet50

Chương 2: Mô tả dữ liệu và tiền xử lý dữ liệu:

1. Mô tả dữ liệu:

Tập dữ liệu được sử dụng là Chicken Disease Dataset:

<https://www.kaggle.com/datasets/allandclive/chicken-disease-1>

Tập dữ liệu bao gồm tập Train 8067 ảnh màu của phân gà và file train_data.csv lưu label của từng ảnh, được chia thành 4 label, cụ thể:

✓ Coccidiosis: bệnh cầu trùng ở gà

Độ loăng không nhất quán — có lúc giống loăng vừa, có lúc đặc sệt.

Màu thường chuyển nâu đất/ nâu vàng sẫm hơn

Phân bố không đồng đều

Bề mặt nền sần sùi, không sạch

✓ Healthy: gà khỏe mạnh

Phân dạng rắn, rõ khuôn, không loăng quá.

Màu thường nâu/nâu sẫm, có chút vệt trắng bình thường.

Ít khi có bọt hoặc dịch loăng.

Đa phần ảnh đủ sáng, nền sạch.

✓ New Castle Disease: bệnh new castle

Độ loăng không nhất quán — có lúc giống loăng vừa, có lúc đặc sệt.

Màu thường chuyển nâu đất/ nâu vàng sẫm hơn

Phân bố không đồng đều

Bề mặt nền sần sùi, không sạch

✓ Salmonella: bệnh nhiễm khuẩn

Kết cấu hơi đặc, tích thành vệt trên bề mặt

Da phần có màu nâu sẫm/xám, nhiều màu trắng, dễ nhầm sang healthy

Bề mặt có màu xanh dương

Các hình ảnh trong tập dữ liệu có sự đa dạng về:

- ✓ Kích thước không đồng nhất
- ✓ Điều kiện ánh sáng khác nhau
- ✓ Góc chụp đa dạng
- ✓ Nền ảnh phức tạp

Điều này giúp mô hình học được đặc trưng tổng quát nhưng cũng làm tăng độ phức tạp của bài toán phân loại.

Chương 3: Phương pháp và mô hình học máy áp dụng

1. Mô hình ResNet50:

1.1 Giới thiệu chung:

a. Khái niệm:

ResNet-50 là một mô hình thị giác máy tính dựa trên một loại mạng nơ-ron gọi là Mạng nơ-ron tích chập (CNN) . Mạng nơ-ron tích chập được thiết kế để giúp máy tính hiểu thông tin thị giác bằng cách học các mẫu hình trong hình ảnh, chẳng hạn như các cạnh, màu sắc hoặc hình dạng, và sử dụng các mẫu hình đó để nhận dạng và classify các vật thể.

Một tính năng chính của ResNet-50 là việc sử dụng các kết nối dư, còn được gọi là kết nối tắt. Đây là những đường dẫn đơn giản cho phép mô hình bỏ qua một số bước trong quá trình học. Nói cách khác, thay vì buộc mô hình phải truyền thông tin qua mọi lớp, các đường tắt này cho phép nó chuyển tiếp các chi tiết quan trọng một cách trực tiếp hơn. Điều này làm cho việc học nhanh hơn và đáng tin cậy hơn.

b. Cơ chế hoạt động:

- Trích xuất đặc trưng cơ bản: Mô hình bắt đầu bằng cách áp dụng một phép toán gọi là tích chập (convolution). Điều này bao gồm trượt các bộ lọc nhỏ (gọi là kernel) trên hình ảnh để tạo ra các bản đồ đặc trưng (feature maps) - các phiên bản mới của hình ảnh làm nổi bật các mẫu cơ bản như cạnh hoặc kết cấu. Đây là cách mô hình bắt đầu nhận biết thông tin trực quan hữu ích.

- Học các đặc trưng phức tạp: Khi dữ liệu di chuyển qua mạng, kích thước của các bản đồ đặc trưng sẽ nhỏ hơn. Điều này được thực hiện thông qua các kỹ thuật như gộp nhóm (pooling) hoặc sử dụng các bộ lọc có bước lớn hơn (gọi là strides). Đồng thời, mạng tạo ra nhiều bản đồ đặc trưng hơn, giúp nó nắm bắt các mẫu ngày càng phức tạp, như hình dạng, bộ phận của đối tượng hoặc kết cấu.

- Nén và mở rộng dữ liệu: Mỗi giai đoạn nén dữ liệu, xử lý dữ liệu đó và sau đó mở rộng trở lại. Điều này giúp mô hình học hỏi trong khi tiết kiệm bộ nhớ.

- Kết nối tắt: Đây là những đường dẫn đơn giản cho phép thông tin bỏ qua các lớp thay vì đi qua từng lớp. Chúng giúp cho việc học trở nên ổn định và hiệu quả hơn.

Đưa ra dự đoán: Ở cuối mạng, tất cả thông tin đã học được kết hợp và truyền qua một hàm softmax. Hàm này xuất ra một phân phối xác suất trên các lớp có thể có, cho biết độ tin cậy của mô hình trong mỗi dự đoán—ví dụ: 90% mèo, 9% chó, 1% ô tô.

c. Cấu trúc:

ResNet50 được xây dựng từ nhiều Residual Block (phần dư) xếp chồng lên nhau. Mỗi block trong ResNet50 sử dụng kiến trúc Bottleneck (làm Residual Block chạy nhanh hơn bằng cách bóp nhỏ dữ liệu rồi nói ra lại)

d. Feature Extraction:

Trong giai đoạn Feature Extraction, toàn bộ các lớp trích xuất đặc trưng của ResNet50 được đóng băng (freeze), tức là các trọng số của chúng không được cập nhật trong quá trình huấn luyện. Việc đóng băng này nhằm giữ nguyên các kiến thức đã học từ ImageNet, tránh làm sai lệch các đặc trưng tổng quát vốn đã được tối ưu rất tốt.

Sau khi đóng băng backbone ResNet50, các lớp phân loại mới được gắn thêm vào phía sau mô hình, bao gồm lớp Global Average Pooling để biến đổi feature map thành vector đặc trưng, lớp Batch Normalization để ổn định phân phối dữ liệu, lớp Dropout để giảm hiện tượng overfitting và lớp Dense với hàm kích hoạt Softmax để phân loại các loại bệnh gà.

e. Fine - turning:

Các lớp ở đầu mạng tiếp tục được đóng băng do chúng chỉ học các đặc trưng chung như cạnh và màu sắc, không cần học lại. Chỉ một số lớp cuối được phép cập nhật trọng số để học các đặc trưng phức tạp và đặc thù của bộ dữ liệu đang xét.

1.2 Source Code:

a. Thư viện và API sử dụng:

`import os`: Thư viện `os` được sử dụng để tương tác với hệ điều hành, chủ yếu nhằm ghép và quản lý đường dẫn tệp ảnh trong quá trình đọc dữ liệu từ thư mục.

`import numpy as np`: NumPy là thư viện cốt lõi của Python dùng để xử lý mảng số và thực hiện các phép toán số học hiệu năng cao, đặc biệt quan trọng trong việc thao tác dữ liệu đầu vào và đầu ra của mô hình học sâu.

`import pandas as pd`: Pandas được sử dụng để đọc và xử lý dữ liệu dạng bảng từ tệp CSV, hỗ trợ các thao tác lọc, thống kê và quản lý nhãn dữ liệu trước khi đưa vào mô hình.

`import matplotlib.pyplot as plt`: Matplotlib là thư viện dùng để trực quan hóa dữ liệu, được sử dụng trong nghiên cứu này để vẽ các biểu đồ độ chính xác, hàm mất mát và hiển thị kết quả dự đoán hình ảnh.

`from sklearn.model_selection import train_test_split`: Hàm `train_test_split` được dùng để chia tập dữ liệu ban đầu thành tập huấn luyện và tập kiểm định, giúp đánh giá khả năng tổng quát hóa của mô hình.

`from sklearn.metrics import confusion_matrix, classification_report`: Các hàm này dùng để đánh giá chi tiết hiệu năng mô hình thông qua ma trận nhầm lẫn và các chỉ số như Precision, Recall và F1-score cho từng lớp.

`import tensorflow as tf`: TensorFlow là framework học sâu do Google phát triển, cung cấp nền tảng để xây dựng, huấn luyện và đánh giá các mô hình mạng nơ-ron sâu trên CPU hoặc GPU.

`from tensorflow.keras import layers, models`: Module layers cung cấp các lớp xây dựng mạng nơ-ron như Dense, Conv2D, BatchNormalization và Dropout, trong khi models hỗ trợ ghép các lớp này thành một mô hình hoàn chỉnh.

`from tensorflow.keras.preprocessing.image import ImageDataGenerator`: ImageDataGenerator được sử dụng để đọc ảnh theo từng batch, chuẩn hóa dữ liệu và thực hiện các kỹ thuật tăng cường dữ liệu (Data Augmentation) nhằm cải thiện khả năng tổng quát hóa của mô hình.

`from tensorflow.keras.applications import ResNet50`: ResNet50 là kiến trúc mạng nơ-ron tích chập sâu được sử dụng làm backbone trong nghiên cứu này, cho phép trích xuất đặc trưng mạnh mẽ từ hình ảnh nhờ cơ chế Residual Learning.

`from tensorflow.keras.applications.resnet50 import preprocess_input`: Hàm preprocess_input dùng để tiền xử lý ảnh đầu vào theo đúng chuẩn của ResNet50 pretrained trên ImageNet, giúp dữ liệu đầu vào phù hợp với các trọng số đã học sẵn.

```
✓import os #ghép đường dẫn file ảnh
import numpy as np #thao tác mảng số
import pandas as pd #đọc CSV, thao tác bảng
import matplotlib.pyplot as plt # vẽ biểu đồ loss/accuracy và hiển thị ảnh
from sklearn.model_selection import train_test_split #chia dữ liệu train/validation
from sklearn.metrics import confusion_matrix, classification_report #đánh giá chi tiết
import tensorflow as tf # framework deep learning
from tensorflow.keras import layers, models #xây model, train, predict
from tensorflow.keras.preprocessing.image import ImageDataGenerator #đọc ảnh theo batch + augmentation
from tensorflow.keras.applications import ResNet50
from tensorflow.keras.applications.resnet50 import preprocess_input
```

b. EDA

- Tạo biểu đồ so sánh phân bố lớp

```
from .preprocessing import load_dataframe

def run_eda(save_dir="results/figures"):
    os.makedirs(save_dir, exist_ok=True) #tạo folder nếu chưa tồn tại, không lỗi nếu folder đã có sẵn

    df = load_dataframe()

    #So sánh phân bố lớp
    df["label"].value_counts().plot(kind="bar") #đếm số lượng ảnh của mỗi lớp.
    plt.title("Phân bố số lượng ảnh theo lớp")
    plt.xlabel("Lớp")
    plt.ylabel("Số ảnh")
    plt.tight_layout() #tự canh bố cục để chữ không bị cắt.
    out1 = os.path.join(save_dir, "eda_class_distribution.png") #ghép đường dẫn
    plt.savefig(out1, dpi=200) #lưu biểu đồ
    plt.show()
```

- Hiển thị ảnh mẫu:

```
#Hiển thị 1 vài ảnh mẫu
fig = plt.figure(figsize=(8,8))
labels = df["label"].unique()

for i, label in enumerate(labels):
    sample = df[df["label"] == label].sample(1).iloc[0] #lọc ra các dòng thuộc lớp label,
    #chọn ngẫu nhiên 1 dòng trong lớp đó, rồi lấy dòng đầu
    ax = fig.add_subplot(2,2,i+1) #Tạo đúng lưới 2x2 cho 4 lớp.
    ax.imshow(Image.open(sample["filepath"])) #Vẽ ảnh đó lên subplot ax
    ax.set_title(label)
    ax.axis("off") #Tắt trục tọa độ x,y, số

plt.tight_layout() #Tự động căn chỉnh bố cục
out2 = os.path.join(save_dir, "eda_sample_images.png")
plt.savefig(out2, dpi=200)
plt.show()

print("Saved:", out1)
print("Saved:", out2)
```

c. Preprocessing:

- hàm lưu history (lịch sử train)

```
def save_history(history, path):  
    os.makedirs(os.path.dirname(path),  
    with open(path, "w", encoding="utf-  
    json.dump(history.history, f, #  
    ensure_ascii=False,  
    indent=2)  
    print("Saved history to", path) #fi
```

os.path.dirname(path): lấy thư mục cha của file cần lưu.

Ví dụ path="results/history/resnet.json" $\Rightarrow$ dirname là "results/history".

os.makedirs(..., exist_ok=True): tạo thư mục nếu chưa có (đỡ lỗi "No such file or directory").

json.dump(... ensure_ascii=False ...): giữ tiếng Việt không bị mã hóa

indent=2: file json dễ đọc hơn.

- Đường dẫn dữ liệu

```
IMG_DIR    = r"D:/Hocmay/ga/Train" #folder chứa ảnh  
CSV_PATH   = r"D:/Hocmay/ga/train_data.csv" #file csv chứa tên ảnh + label
```

IMG_DIR dùng để chỉ đường dẫn tới thư mục chứa toàn bộ ảnh huấn luyện.

CSV_PATH là đường dẫn tới tệp CSV lưu tên ảnh và nhãn (label) tương ứng, giúp chương trình ánh xạ chính xác giữa ảnh và lớp bệnh.

- Cấu hình ảnh:

```
# ResNet50/EfficientNet chuẩn 224x224, CNN 150x150
IMG_SIZE_RESNET_EFF = (224, 224)
IMG_SIZE_CNN = (150, 150)

BATCH_SIZE = 32          # mỗi lần cập nhật trọng :
SEED = 42                 # giúp kết quả chia train,
```

- Thiết lập seed để cố định tính ngẫu nhiên:

```
tf.random.set_seed(SEED)
np.random.seed(SEED) #Cố
# Nếu ko set mỗi lần chạy
```

- Đọc dữ liệu, tạo đường dẫn tới từng ảnh (ghép đường dẫn ảnh với tên ảnh trong images)

```
def load_dataframe():|
    #Đọc CSV + tạo filepath + lọc ảnh thiếu
    df = pd.read_csv(CSV_PATH)

    # CSV có 2 cột: images, label
    #tạo cột filepath chứa đường dẫn ảnh
    df["filepath"] = df["images"].apply(lambda x: os.path.join(IMG_DIR, x))

    # Lọc ảnh bị thiếu (nếu có)
    df = df[df["filepath"].apply(os.path.exists)].reset_index(drop=True)

    df["label"] = df["label"].astype(str).str.strip() # xóa khoảng trắng thừa nếu có

    print(df.head())
    print("Tổng ảnh:", len(df))
    print("Số lớp:", df["label"].nunique())
    print(df["label"].value_counts())
    return df
```

Trong đó :

- + apply() dùng để áp dụng một hàm cho từng phần tử trong cột.
- + lambda x: là một hàm ẩn danh, trong đó x đại diện cho tên từng file ảnh.
- + os.path.join(IMG_DIR, x) ghép thư mục gốc chứa ảnh với tên file, đảm bảo đúng định dạng đường dẫn trên hệ điều hành.

==> KQ:

	images	label	filepath
0	salmo.1558.jpg	Salmonella	D:/Hocmay/ga/Train/salmo.1558.jpg
1	cocci.1866.jpg	Coccidiosis	D:/Hocmay/ga/Train/cocci.1866.jpg
2	cocci.171.jpg	Coccidiosis	D:/Hocmay/ga/Train/cocci.171.jpg
3	salmo.1484.jpg	Salmonella	D:/Hocmay/ga/Train/salmo.1484.jpg
4	ncd.100.jpg	New Castle Disease	D:/Hocmay/ga/Train/ncd.100.jpg

Tổng ảnh: 8067
Số lớp: 4
label
Salmonella 2625
Coccidiosis 2476
Healthy 2404
New Castle Disease 562
Name: count, dtype: int64

- Ta thấy không có ảnh nào thiếu label trong file csv, New Castle Disease bị lệch tỷ lệ nhiều với các ảnh khác, nhưng ta thử chạy mô hình không xử lý dữ liệu lệch xem như thế nào, dễ bị nhầm lẫn sang lớp nào

- Chia dữ liệu để train:

```
def load_and_split_data(test_size=0.2):  
    #Hàm đọc file csv và chia tập dữ liệu.  
    df = load_dataframe()  
  
    #Chia dữ liệu cho tập train/test  
    train_df, val_df = train_test_split(  
        df, test_size=test_size, random_state=SEED, stratify=df["label"]  
    )  
    #stratify giữ nguyên tỉ lệ các lớp (label) ở cả train và val  
    print("Train:", len(train_df), "Val:", len(val_df))  
    return train_df, val_df
```

d. Feature:

- Khởi tạo ImageDataGenerator cho tập huấn luyện:

```

train_datagen = ImageDataGenerator(
    preprocessing_function=preprocess_input, #chuẩn hoá ảnh theo đúng
    rotation_range=15, #xoay +-15 độ
    width_shift_range=0.1, # Dịch ngang lên đến 10% chiều rộng ảnh -
    height_shift_range=0.1, # Dịch dọc lên đến 10% chiều cao ảnh -->
    zoom_range=0.1, # Phóng to/thu nhỏ ảnh trong khoảng ±20%
    horizontal_flip=True #Lật ngang --> Hữu ích cho ảnh đối xứng
)

```

Trong đó:

ImageDataGenerator(...) là một class của Keras dùng để cấu hình cách đọc và biến đổi ảnh.

preprocessing_function=preprocess_input chỉ định hàm tiền xử lý chuẩn của ResNet50 pretrained trên ImageNet

rotation_range=15 cho phép xoay ảnh ngẫu nhiên trong khoảng 15 độ.

width_shift_range=0.1 dịch ảnh ngẫu nhiên theo chiều ngang tối đa 10% chiều rộng.

height_shift_range=0.1 dịch ảnh ngẫu nhiên theo chiều dọc tối đa 10% chiều cao.

zoom_range=0.1 phóng to hoặc thu nhỏ ảnh trong khoảng 10%.

horizontal_flip=True lật ngang ảnh ngẫu nhiên, hữu ích với các đối tượng có tính đối xứng.

- ImageDataGenerator cho tập validation:

```

#validation KHÔNG dùng augmentation do phải di
val_datagen = ImageDataGenerator(
    preprocessing_function=preprocess_input
)

```

Tập validation không sử dụng augmentation vì cần phản ánh đúng dữ liệu thực tế ngoài đời. Việc chỉ áp dụng preprocess_input đảm bảo ảnh validation được xử lý giống ảnh train về mặt chuẩn hóa nhưng không bị biến đổi hình học.

- Tạo generator cho tập huấn luyện:

```
# Tạo generator cho tập huấn luyện
train_gen = train_datagen.flow_from_dataframe(
    train_df,
    x_col="filepath",
    y_col="label",
    target_size=IMG_SIZE,
    class_mode="categorical", #nhãn được one-hot (vd [0,0,1,0])
    batch_size=BATCH_SIZE,
    shuffle=True, #để model không học theo thứ tự cố định
    seed=SEED
)
```

Đoạn lệnh này tạo ra train_gen, một generator sẽ tự động đọc ảnh từ DataFrame, tiền xử lý, tăng cường dữ liệu và đưa ảnh vào mô hình:

train_df là DataFrame chứa đường dẫn ảnh và nhãn.

class_mode="categorical" chuyển nhãn thành vector one-hot (phù hợp với softmax).

shuffle=True trộn ngẫu nhiên dữ liệu để mô hình không học theo thứ tự cố định.

seed=SEED đảm bảo việc shuffle có tính lặp lại.

- Tạo generator cho tập validation:

```
# Tạo generator cho tập validation
val_gen = val_datagen.flow_from_dataframe(
    val_df,
    x_col="filepath",
    y_col="label",
    target_size=IMG_SIZE,
    class_mode="categorical",
    batch_size=BATCH_SIZE,
    shuffle=False #Giữ nguyên thứ tự ảnh trong validation
)
```


val_gen dùng để đánh giá mô hình trên tập validation. Dữ liệu validation không bị trộn để đảm bảo thứ tự ảnh và nhãn luôn khớp, đặc biệt quan trọng khi vẽ confusion matrix và classification report.

e. Huấn luyện mô hình:

- Xây dựng mô hình:

```
#Load ResNet50 pretrained + đóng  
base_model = ResNet50(  
    weights="imagenet", #lấy tr  
    include_top=False, #bỏ phần  
    input_shape=(*IMG_SIZE, 3)  
)  
base_model.trainable = False #
```

Đoạn mã này khởi tạo mô hình ResNet50 đã được huấn luyện sẵn trên tập ImageNet.

weights="imagenet" nạp trọng số pretrained từ ImageNet.

include_top=False loại bỏ lớp Dense 1000 neuron của ImageNet.

input_shape=(*IMG_SIZE, 3) xác định ảnh đầu vào có 3 kênh màu RGB.

base_model.trainable = False khóa toàn bộ trọng số của backbone.

```
inputs = layers.Input(shape=(*IMG_SIZE, 3))
```

layers.Input() tạo một tensor đầu vào.

shape xác định kích thước ảnh, không bao gồm batch size.

```
x = base_model(inputs, training=False)
```

Ảnh đầu vào được đưa qua ResNet50 để trích xuất đặc trưng (feature maps)

training=False ép các lớp như BatchNorm không cập nhật thông kê trong lúc train.

```
x = layers.GlobalAveragePooling2D()(x)

x = layers.BatchNormalization()(x) #Cl
```

ResNet50 trả về feature map có dạng (H, W, C), ví dụ (7, 7, 2048). Lớp Global Average Pooling sẽ lấy giá trị trung bình trên toàn bộ không gian H×W cho từng kênh, biến feature map thành một vector đặc trưng gọn có kích thước 2048. Điều này giúp giảm số tham số và hạn chế overfitting.

Lớp Batch Normalization chuẩn hóa các giá trị trong vector đặc trưng về cùng thang đo

```
x = layers.Dropout(0.4)(x)
```

Dropout ngẫu nhiên vô hiệu hóa 40% số neuron trong mỗi lần huấn luyện, buộc mô hình không phụ thuộc quá nhiều vào một số đặc trưng cụ thể.

```
#softmax biến 3 số đó thành xác suất (tổng = 1)
outputs = layers.Dense(NUM_CLASSES, activation="softmax")(x)

model = models.Model(inputs, outputs)#Tạo model với đầu vào là
model.summary() #in ra kiến trúc: số layer, shape từng tầng, ...
```

Lớp Dense cuối cùng đóng vai trò phân loại. Hàm kích hoạt Softmax biến đầu ra thành các xác suất, trong đó tổng xác suất của tất cả các lớp bằng 1. Mô hình sẽ chọn lớp có xác suất cao nhất làm kết quả dự đoán.

- Cấu hình huấn luyện mô hình:

```
model.compile(
    optimizer=tf.keras.optimizers.Adam(1e-3),
    loss="categorical_crossentropy", #Là thước
    metrics=["accuracy"]
)
```

optimizer=Adam(1e-3): Adam là thuật toán tối ưu dùng để cập nhật trọng số của mô hình sau mỗi lần dự đoán sai. Learning rate 0.001 phù hợp cho giai đoạn Feature Extraction, khi chỉ huấn luyện phần head mới và cần tốc độ học tương đối nhanh.

loss="categorical_crossentropy": Hàm mất mát dùng cho bài toán phân loại đa lớp với nhãn ở dạng one-hot. Hàm loss này đo mức độ sai lệch giữa xác suất dự đoán của mô hình và nhãn thật.

metrics=["accuracy"]: Chỉ số accuracy được dùng để theo dõi độ chính xác của mô hình trong quá trình huấn luyện và đánh giá.

- Lưu mô hình tốt nhất:

```
callbacks = [  
    tf.keras.callbacks.ModelCheckpoint( #Tự động lưu model tốt nhất trong lúc train  
        "resnet50_best.keras", #File model tốt nhất (có thể load lại sau)  
        monitor="val_accuracy", #Theo dõi accuracy trên validation  
        save_best_only=True, #Chỉ lưu khi val_accuracy tăng  
        verbose=1  
    ),  
]
```

resnet50_best.keras: tên file lưu mô hình tốt nhất.

monitor=val_accuracy: theo dõi độ chính xác trên tập validation.

save_best_only=True: chỉ lưu khi val_accuracy tăng lên so với các epoch trước.

verbose=1: in thông báo mỗi khi mô hình được lưu.

- Dừng huấn luyện sớm:

```
tf.keras.callbacks.EarlyStopping(  
    monitor="val_accuracy",  
    patience=3, #Nếu 3 epoch liên  
    restore_best_weights=True #Sai  
)
```

monitor="val_accuracy": theo dõi độ chính xác validation.

patience=3: nếu 3 epoch liên tiếp không cải thiện thì dừng huấn luyện.

restore_best_weights=True: sau khi dừng, mô hình sẽ quay lại bộ trọng số tốt nhất

- Giảm learning rate khi mô hình không còn cải thiện về mặt loss trên tập validation:

```
#Model học tốt lúc đầu, sau đó val_loss không giảm nữa, có thể  
tf.keras.callbacks.ReduceLROnPlateau( #giảm LR  
    monitor="val_loss",  
    factor=0.5, #Learning rate mới = learning rate cũ × 0.5  
    patience=2,  
    verbose=1  
)
```

monitor=val_loss: theo dõi hàm mất mát trên validation.

factor=0.5: learning rate mới bằng một nửa learning rate cũ.

patience=2: nếu 2 epoch liên tiếp val_loss không giảm thì giảm learning rate.

verbose=1: in thông báo khi learning rate bị giảm.

- Feature Extraction:

```
history_fe = model.fit(  
    train_gen,  
    validation_data=val_gen,  
    epochs=EPOCHS_FE,  
    callbacks=callbacks,  
)  
# accuracy - độ chính xác trên tập huấn luyện (training set)  
# loss - giá trị hàm mất mát (loss function) trên tập huấn luyện  
# val_accuracy - độ chính xác trên tập kiểm định (validation set)  
# val_loss - giá trị mất mát trên tập validation
```

train_gen: Generator cung cấp dữ liệu huấn luyện (ảnh + nhãn) theo từng batch, có augmentation.

validation_data=val_gen: Generator cung cấp dữ liệu validation, dùng để đánh giá khả năng tổng quát hóa của mô hình sau mỗi epoch.

epochs=EPOCHS_FE: Số vòng lặp huấn luyện cho giai đoạn Feature Extraction. Mỗi epoch tương ứng với việc mô hình “nhìn” toàn bộ tập train một lần.

callbacks=callbacks: Các cơ chế hỗ trợ như lưu mô hình tốt nhất, dừng sớm và giảm learning rate.

- Fine-Tuning:

```
#Fine-tuning = mở một phần ResNet50 để học tiếp  
base_model.trainable = True #Cho phép ResNet50  
  
# Chỉ mở 30 layer cuối  
fine_tune_at = len(base_model.layers) - 30  
for layer in base_model.layers[:fine_tune_at]:  
    layer.trainable = False
```

Mở khóa backbone để cho phép cập nhật trọng số, nhận 30 layers cuối

- Compile lại mô hình với learning rate nhỏ:

```
#giảm learning rate xuống 1e-5 do:  
# Feature Extraction: Train từ đầu head cần học nhanh  
# Fine-tuning: Chỉ chỉnh nhẹ các layer đã học sẵn  
model.compile(  
    optimizer=tf.keras.optimizers.Adam(1e-5),  
    loss="categorical_crossentropy",  
    metrics=["accuracy"]  
)
```

- Huấn luyện giai đoạn Fine-tuning:

```
history_ft = model.fit(  
    train_gen,  
    validation_data=val_gen,  
    epochs=EPOCHS_FT,  
    callbacks=callbacks,  
)
```


- lưu TOÀN BỘ mô hình vào file và Load lại mô hình đã lưu

```
model.save("resnet50_final.keras")
loaded = tf.keras.models.load_model("resnet50_final.keras")
loaded.evaluate(val_gen) #Đánh giá mô hình trên validation [loss, accuracy]
```

f. Evaluation:

- Đọc history JSON của model:

```
def _safe_load_history(path):
    if path is None:
        return None
    if not os.path.exists(path):
        print(f"Không thấy history: {path}")
        return None
    with open(path, "r", encoding="utf-8") as f:
        return json.load(f)
```

- Kiểm tra có dữ liệu không, nếu có thì nối 2 history FE và FT:

```
def plot_history_from_json(history_fe_json, history_ft_json=None,
                           title_prefix="Model", save_path=None):
    #Vẽ biểu đồ từ history JSON (để evaluation chạy độc lập, không cần train lại)
    #Kiểm tra có dữ liệu không
    if history_fe_json is None:
        print("Không có history_fe_json để vẽ.")
        return

    # Nếu có FT thì nối
    if history_ft_json is not None:
        def merge_histories(h1, h2):
            out = {}
            for k in h1.keys():
                out[k] = list(h1.get(k, [])) + list(h2.get(k, []))
            return out

        hist = merge_histories(history_fe_json, history_ft_json)
        len_fe = len(history_fe_json.get("accuracy", [])) # Điểm đánh dấu kết thúc giai đoạn 1
    else:
        hist = history_fe_json
        len_fe = None
```

+ out = {} : tạo dict rỗng.

+ for k in h1.keys():

h1.keys() trả về danh sách key của dict (ví dụ: "accuracy", "loss", ...)

k là từng key.

+ h1.get(k, [])

.get(key, default) lấy value theo key

nếu key không có , trả [] (tránh lỗi KeyError)

+ list(...) ép về list (để chắc chắn nối được)

+ nối 2 list: list_FE + list_FT ==> list dài hơn

+ return out trả dict đã nối.

- Vẽ biểu đồ:

```
plt.figure(figsize=(14, 5))

# Accuracy
#tạo lưới subplot 1 hàng, 2 cột, chọn ô thứ 1
plt.subplot(1, 2, 1)
#vẽ train
plt.plot(hist.get("accuracy", []), #Lấy danh sách accuracy theo epoch
         label="Train Accuracy",
         marker='o', #Mỗi epoch được vẽ một chấm tròn
         markersize=3)
#vẽ val
plt.plot(hist.get("val_accuracy", []), label="Val Accuracy", marker='o', markersize=3)
#vẽ vạch đỏ phân FE và FT
if len_fe is not None and len_fe > 0: #
    plt.axvline(x=len_fe - 1, #vẽ đường thẳng đứng, -1 vì epoch bắt đầu vẽ từ 0
               color='red',
               linestyle='--',
               label='Start Fine-tuning')

plt.title(f"{title_prefix} - Accuracy")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend(loc="lower right")
plt.grid(True, alpha=0.3)
```

```
# Loss
#tạo lưới subplot 1 hàng, 2 cột, chọn ô thứ 2
plt.subplot(1, 2, 2)
plt.plot(hist.get("loss", []), label="Train Loss", marker='o', markersize=3)
plt.plot(hist.get("val_loss", []), label="Val Loss", marker='o', markersize=3)
if len_fe is not None and len_fe > 0:
    plt.axvline(x=len_fe - 1, color='red', linestyle='--', label='Start Fine-tuning')
plt.title(f"{title_prefix} - Loss")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend(loc="upper right")
plt.grid(True, alpha=0.3)

plt.tight_layout() #Tự động căn chỉnh khoảng cách, nếu ko có thì title đề lên subplot
```

- Trong đó:

- + hist.get("accuracy", []) lấy list y.
- + x mặc định là 0..n-1
- + label=: tên hiện trên legend
- + marker='o': vẽ điểm tròn mỗi epoch
- + markersize=3: điểm nhỏ
- + axvline : đường thẳng đứng.
- + x=len_fe - 1: vì epoch index bắt đầu từ 0, nếu FE 10 epoch , epoch cuối là 9
- + linestyle='--' nét đứt.

- Lưu biểu đồ nếu truyền đường dẫn:

```
if save_path is not None: #nếu có truyền đường dẫn lưu file ảnh
    os.makedirs(os.path.dirname(save_path), exist_ok=True)
    plt.savefig(save_path, dpi=200) #Lưu hình vẽ hiện tại ra file
    print("Saved figure:", save_path)

plt.show()
```

- Tạo Confusion Matrix và vẽ heatmap:


```
#tạo Confusion Matrix và vẽ heatmap
def evaluate_confusion_report(model, val_gen, save_dir="results/figures", prefix="model"):
    os.makedirs(save_dir, exist_ok=True)

    #dự đoán trên validation
    val_gen.reset() #đưa con trỏ về đầu
    pred_probs = model.predict(val_gen) #xác suất dự đoán cho mỗi lớp
    y_pred = np.argmax(pred_probs, axis=1) #Chọn lớp có xác suất lớn nhất cho từng ảnh
    y_true = val_gen.classes #Nhãn thật (ground truth) đã được generator gán sẵn

    # mapping index -> label
    idx_to_label = {v: k for k, v in val_gen.class_indices.items()}
    labels_sorted = [idx_to_label[i] for i in range(len(idx_to_label))]
```

model: model Keras đã load (tf.keras.models.load_model(...))

val_gen: generator validation (ImageDataGenerator / flow_from_dataframe /)

save_dir="results/figures": thư mục lưu ảnh confusion matrix (mặc định)

prefix="model": tiền tố đặt tên file (ví dụ: resnet50_confusion_matrix.png)

```
#dự đoán trên validation
val_gen.reset() #đưa con trỏ về đầu
pred_probs = model.predict(val_gen) #xác suất dự đoán cho mỗi lớp
y_pred = np.argmax(pred_probs, axis=1) #Chọn lớp có xác suất lớn nhất cho từng ảnh
y_true = val_gen.classes #Nhãn thật (ground truth) đã được generator gán sẵn
```

val_gen.reset() đưa generator về đầu để đảm bảo thứ tự ảnh khi predict là đúng (tránh lệch nhãn).

pred_probs = model.predict(val_gen) trả về xác suất dự đoán cho mỗi ảnh theo từng lớp, ví dụ [0.1, 0.7, 0.2, 0.0].

np.argmax(..., axis=1) lấy chỉ số lớp có xác suất cao nhất cho từng ảnh → chính là nhãn dự đoán.

val_gen.classes là danh sách nhãn thật (mã số 0,1,2,3...) do generator tự gán sẵn theo thứ tự ảnh.

```
# mapping index -> label
idx_to_label = {v: k for k, v in val_gen.class_indices.items()} #Đảo dict để map index ==> label
labels_sorted = [idx_to_label[i] for i in range(len(idx_to_label))]
```

`len(idx_to_label)` = số lớp (vd 4)

`range(4)` → 0,1,2,3

lấy `idx_to_label[i]` theo thứ tự

- Vẽ biểu đồ:

```
#Tính Confusion Matrix
cm = confusion_matrix(y_true, y_pred)

plt.figure(figsize=(10, 8))
sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    cmap='Blues',
    xticklabels=labels_sorted,
    yticklabels=labels_sorted
)

plt.title('Confusion Matrix', fontsize=16)
plt.xlabel('Predicted Label')
plt.ylabel('True Label')
plt.tight_layout()
```

- Lưu biểu đồ và in ra report

```
#Lưu hình và hiển thị
out_cm = os.path.join(save_dir, f"{prefix}_confusion_matrix.png") #Tạo đường dẫn file output
plt.savefig(out_cm, dpi=200)
plt.show()
print("Saved:", out_cm)

# Classification report
print(classification_report(y_true, y_pred, target_names=labels_sorted))
```

- Chạy độc lập evaluation mà không cần train lại model:

```

#chạy độc lập, chỉ load model
def run_evaluation(model_name="resnet50"):
    #Tạo thư mục lưu hình
    save_dir = "results/figures"
    os.makedirs(save_dir, exist_ok=True)

```

- Load model từ file keras, history:

```

if model_name == "resnet50":
    #Khai báo đường dẫn file model và history
    model_path = "models/resnet50_final2.keras"
    hist_fe_path = "results/history/resnet50_history_fe.json"
    hist_ft_path = "results/history/resnet50_history_ft.json"

    from .preprocessing import load_and_split_data
    from .feature import get_val_generator_resnet50

    #Tách dữ liệu train/val
    train_df, val_df = load_and_split_data()
    #Tạo generator validation cho ResNet50
    val_gen = get_val_generator_resnet50(val_df)

    #load model
    loaded = tf.keras.models.load_model(model_path, compile=False)

    # Vẽ history từ JSON
    hfe = _safe_load_history(hist_fe_path)
    hft = _safe_load_history(hist_ft_path)
    plot_history_from_json(hfe, hft, title_prefix="ResNet50",
                           save_path=os.path.join(save_dir, "resnet50_history.png"))

    # Confusion matrix + report
    evaluate_confusion_report(loaded, val_gen, save_dir=save_dir, prefix="resnet50")

```

model_path: file model đã train xong, lưu dạng .keras

hist_fe_path: history giai đoạn Feature Extraction (FE)

hist_ft_path: history giai đoạn Fine-tuning (FT)

2. Mô hình EfficientNetB0:

2.1 Giới thiệu chung.

a. *Khái niệm*

- EfficientNetB0 là một mô hình mạng nơ-ron tích chập (CNN), phiên bản cơ sở (baseline) của dòng mô hình EfficientNet, được giới thiệu bởi Google vào năm 2019. Đây là một bước đột phá trong thị giác máy tính vì nó đạt được độ chính xác rất cao nhưng lại nhẹ và nhanh hơn nhiều so với các kiến trúc truyền thống như ResNet hay MobileNet.
- Có hai dòng chính: EfficientNet V1 và EfficientNetV2
 - + **EfficientNet V1** có 8 phiên bản từ B0 đến B7 được phát triển bằng cách mở rộng quy mô từ phiên bản gốc B0 theo nguyên tắc "Compound Scaling" (tăng đồng thời chiều sâu, chiều rộng và độ phân giải).
 - + **EfficientNetV2** được Google giới thiệu vào năm 2021, tập trung vào việc tăng tốc độ huấn luyện và hiệu quả tham số hơn nữa. Gồm có các bản
 - **V2-S (Small):** Nhỏ gọn, huấn luyện rất nhanh.
 - **V2-M (Medium):** Cân bằng giữa độ chính xác và tốc độ.
 - **V2-L (Large):** Kích thước lớn, cạnh tranh với các mô hình Vision Transformer (ViT).
 - **V2-XL:** Phiên bản cực lớn cho các tác vụ đặc biệt.

b. *Cách hoạt động:*

- Cách hoạt động của EfficientNetB0 là sự kết hợp tinh vi giữa Tìm kiếm kiến trúc thần kinh (NAS) để tạo ra bộ khung tối ưu và Cơ chế chú ý (Attention) để lọc thông tin.
- 3 cơ chế vận hành chính:
 - **Khối xây dựng MBConv (Mobile Inverted Bottleneck)**
MBConv hoạt động theo quy trình 3 bước:

- + **Expansion (Mở rộng):** Sử dụng 1x1 Convolution để tăng số lượng kênh (channels) lên cao (thường gấp 6 lần). Điều này tạo không gian rộng hơn để mô hình học các đặc trưng phức tạp.
- + **Depthwise Convolution:** Thực hiện tích chập riêng biệt trên từng kênh. Đây là chìa khóa giúp EfficientNet cực kỳ nhẹ vì nó giảm lượng tính toán đi nhiều lần so với tích chập truyền thống.
- + **Projection (Nén):** Dùng 1x1 Convolution để nén số kênh quay trở lại kích thước nhỏ hơn để giảm tải cho các lớp sau.
- **Cơ chế Squeeze-and-Excitation (SE)**
 Bên trong mỗi khối MBConv có một module SE hoạt động như một bộ lọc thông minh:
 - + **Squeeze (Nén):** Nó nén toàn bộ thông tin không gian của một kênh ảnh thành một con số duy nhất (thông qua Global Average Pooling).
 - + **Excitation (Kích hoạt):** Nó học cách gán "trọng số" cho từng kênh. Kênh nào chứa thông tin quan trọng (ví dụ: hình dáng con vật) sẽ được phóng đại, kênh nào chứa nhiễu sẽ bị giảm đi.
- **Nguyên lý Compound Scaling (Tỉ lệ hóa hỗn hợp)**
 EfficientNet sử dụng một hệ số toán học ϕ để tăng đồng thời:
 - Chiều sâu (d):** $d = \alpha^\phi$ (Thêm nhiều lớp hơn).
 - Chiều rộng (w):** $w = \beta^\phi$ (Thêm nhiều kênh hơn).
 - Độ phân giải (r):** $r = \gamma^\phi$ (Dùng ảnh đầu vào to hơn).
 Với EfficientNet-B0, giá trị $\phi = 0$. Khi tăng ϕ mô hình tự động mở rộng đều cả 3 hướng, giúp nó luôn đạt hiệu suất tối ưu mà không bị "lệch" về bất kỳ chiều nào.

c. Phương pháp.

- Sử dụng kết hợp **Transfer learning** với **EfficientNetB0**.
- **Transfer learning** là lấy một mô hình đã học từ hàng triệu hình ảnh (như **EfficientNet-B0** học trên tập **ImageNet**). Mô hình này đã biết cách nhận diện

các đặc điểm cơ bản như đường kẻ, góc cạnh, màu sắc. Sau đó chỉ cần dạy nó cách áp dụng những kiến thức đó vào dữ liệu riêng của mình.

- Quy trình:
 - Chọn mô hình tiền huấn luyện (Pre-trained model): Chọn một mô hình mạnh mẽ đã được huấn luyện sẵn (như EfficientNet, ResNet, hoặc MobileNet).
 - Đóng băng (Freezing): Giữ nguyên các lớp trích xuất đặc trưng của mô hình đó để kiến thức cũ không bị mất đi, chỉ thay thế lớp cuối cùng (Output layer) bằng các lớp mới phù hợp với số lượng lớp (classes) trong bài toán.
 - Huấn luyện (Training): Chỉ huấn luyện các lớp mới này trên tập dữ liệu của mình.

2.2 Source code

a. Thư viện và API được sử dụng.

import tensorflow as tf: là một thư viện học sâu (Deep Learning) do Google phát triển dùng để xây dựng mô hình Machine Learning / Deep Learning, huấn luyện và đánh giá mô hình (CNN, RNN, Transformer, EfficientNet, ...). Chạy trên CPU, GPU, TPU.

from tensorflow import keras: là API cấp cao dùng để xây dựng và huấn luyện mô hình Deep Learning.

from tensorflow.keras import layers, models:

+ layers (Các lớp): Chứa các thành phần xây dựng nên mạng nơ-ron như:

+ Dense: Lớp kết nối đầy đủ.

+ Conv2D: Lớp tích chập (dành cho xử lý ảnh).

+ MaxPooling2D: Lớp giảm kích thước ảnh.

+ models (Mô hình): Cung cấp các cấu trúc để ghép các lớp lại với nhau.

from tensorflow.keras.preprocessing.image import ImageDataGenerator:

ImageDataGenerator là một class nằm trong module **preprocessing.image** dùng để đọc ảnh từ thư mục, chuẩn hóa ảnh, tăng cường dữ liệu (Data Augmentation).

import matplotlib.pyplot as plt: là một thư viện của Python dùng để vẽ biểu đồ, đồ thị, hình ảnh. pyplot là module con của matplotlib cung cấp các hàm vẽ theo kiểu MATLAB.

import numpy as np: là một thư viện của Python chuyên xử lý mảng số học (array) và tính toán số học hiệu năng cao

import warnings: là module chuẩn của Python dùng để quản lý và ẩn các cảnh báo khi chạy chương trình.

import os: cho phép Python tương tác với các hệ điều hành..

b. Cấu trúc file.

Với mục đích để dễ dàng bảo trì và sửa lỗi, có khả năng tái sử dụng, mô hình được chia thành các file:

preprocessing.py: Chứa các hàm xử lý dữ liệu thô như: quản lý và liên kết tập dữ liệu, phân chia tập dữ liệu, chuẩn hóa hình ảnh, cơ chế batch processing.

feature_engineering.py: Tập trung vào việc trích xuất hoặc biến đổi các đặc trưng quan trọng từ dữ liệu.

model_EfficientNetB0.py: Xây dựng và tiến hành huấn luyện mô hình.

evaluation.py: vẽ các biểu đồ, ma trận và đánh giá mô hình.

c. preprocessing.py

```
# Cấu hình dữ liệu
IMG_DIR = r"D:\Machinelearning\Bai_tap_nhom\Train_data\Train" # Đường dẫn tới folder chứa ảnh
CSV_PATH = r"D:\Machinelearning\Bai_tap_nhom\Train_data\train_data.csv" # Đường dẫn tới file CSV có nhãn
IMG_SIZE = (224, 224) # Kích thước ảnh đầu vào chuẩn cho mô hình EfficientNetB0
BATCH_SIZE = 32 # Số lượng ảnh mô hình xử lý trong một lần lặp
SEED = 42 # Cố định kết quả ngẫu nhiên (giúp kết quả train luôn giống nhau)
```

IMG_DIR: Đường dẫn thư mục hình ảnh: Xác định vị trí lưu trữ các tệp tin ảnh phân gà (.jpg, .png) trên ổ đĩa. Đây là nguồn dữ liệu thô để mô hình truy xuất trong quá trình huấn luyện.

CSV_PATH: Đường dẫn tệp nhãn dữ liệu: Chỉ định file định dạng .csv chứa danh sách tên tệp ảnh và nhãn bệnh tương ứng. Đây là "chìa khóa" để mô hình biết mỗi tấm ảnh thuộc loại bệnh nào.

IMG_SIZE: Kích thước chuẩn hóa (224x224): Mọi ảnh đầu vào sẽ được đưa về cùng một kích thước. Đây là cấu hình tối ưu và bắt buộc khi sử dụng kiến trúc EfficientNetB0, giúp cân bằng giữa độ chi tiết và tốc độ tính toán.

BATCH_SIZE: Kích thước lô (32 ảnh): Số lượng mẫu dữ liệu được nạp vào bộ nhớ trong một lần lặp huấn luyện. Con số 32 là lựa chọn phổ biến giúp tối ưu hóa hiệu suất của GPU/CPU mà không gây tràn bộ nhớ RAM.

SEED: Cố định các giá trị ngẫu nhiên (như chia dữ liệu, khởi tạo trọng số). Việc này đảm bảo tính tái lập (Reproducibility) - giúp kết quả huấn luyện luôn giống nhau ở mọi lần chạy thử.

```
# Thiết lập Seed cho cả TensorFlow và Numpy để đảm bảo tính tái lập kết quả
tf.random.set_seed(SEED)
np.random.seed(SEED)
```

tf.random.set_seed(SEED): Trọng số của nơ-ron thường được khởi tạo ngẫu nhiên. Dòng lệnh này giúp cố định các giá trị ngẫu nhiên trong thư viện TensorFlow, đảm bảo kiến trúc mô hình được khởi tạo giống nhau ở mọi lần chạy.

np.random.seed(SEED): NumPy được sử dụng để xử lý các mảng dữ liệu và xáo trộn (shuffle) tập dữ liệu. Việc cố định SEED giúp quá trình chia dữ liệu (Train/Val) và các phép biến đổi ma trận luôn cho ra cùng một kết quả duy nhất.

```
# Hàm đọc file csv và chia tập dữ liệu
def load_and_split_data():
    """Đọc CSV và chia tập dữ liệu"""
    df = pd.read_csv(CSV_PATH) # Đọc file CSV chứa tên ảnh và nhãn bệnh
    df['label'] = df['label'].str.strip() # Xóa khoảng trắng thừa ở đầu hoặc cuối tên nhãn
    df['filepath'] = df['images'].apply(lambda x: os.path.join(IMG_DIR, x)) # Tạo đường dẫn đầy đủ cho mỗi tấm ảnh bằng
    df = df[df['filepath'].apply(os.path.exists)].reset_index(drop=True) # Kiểm tra xem ảnh có tồn tại trong máy không,

    train_df, val_df = train_test_split( # Chia dữ liệu: 80% để huấn luyện (Train), 20% để kiểm định (Validation)
        df,
        test_size=0.2, |
        random_state=SEED,
        stratify=df["label"]
    )
    return train_df, val_df
```

Hàm **load_and_split_data** đóng vai trò tiền xử lý cấu trúc dữ liệu trước khi huấn luyện. Quy trình bắt đầu bằng việc chuẩn hóa các nhãn văn bản và thiết lập đường dẫn vật lý cho từng hình ảnh. Kiểm tra sự tồn tại của dữ liệu, chia tập train và validation.

df = pd.read_csv(CSV_PATH): Đọc file quản lý dữ liệu, chuyển đổi danh sách tên ảnh và nhãn từ file CSV thành một DataFrame (cấu trúc dạng bảng) để máy tính dễ xử lý.

df['label'] = df['label'].str.strip(): Xóa khoảng trắng thừa ở đầu hoặc cuối tên nhãn.

df["filepath"] = df["images"].apply(lambda x: os.path.join(IMG_DIR, x)): Chuyển đổi tên file ảnh tương đối trong CSV thành đường dẫn vật lý đầy đủ trên ổ đĩa.

df = df[df["filepath"].apply(os.path.exists)].reset_index(drop=True): Rà soát từng file ảnh trên ổ cứng. Nếu file ảnh bị thiếu hoặc sai tên so với CSV, dòng này sẽ loại bỏ nó.

train_df, val_df = train_test_split(): Chia dữ liệu thành hai phần độc lập: **Train (80%)** để học và **Validation (20%)**.

stratify=df["label"]: Đảm bảo tỉ lệ các lớp bệnh trong tập Train và Validation giống hệt với tỉ lệ trong dữ liệu gốc.

```
# Hàm nạp ảnh từ tập Validation vào mô hình
def get_val_generator(val_df):
    """Tạo generator cho tập validation"""
    val_datagen = ImageDataGenerator(preprocessing_function=preprocess_input)
    return val_datagen.flow_from_dataframe( # Nạp ảnh trực tiếp từ DataFrame
        val_df,
        x_col="filepath", # Cột chứa đường dẫn ảnh
        y_col="label",    # Cột chứa nhãn bệnh
        target_size=IMG_SIZE, # Đưa ảnh về kích thước 224x224
        batch_size=BATCH_SIZE, # Gom ảnh thành các lô 32 tấm
        class_mode="categorical", # Mã hóa nhãn dạng One-hot (đa lớp)
        shuffle=False # Không xáo trộn để vẽ Confusion Matrix
    )
```

Hàm **get_val_generator** đóng vai trò là bộ lọc và nạp dữ liệu chuẩn cho quá trình đánh giá mô hình. Nó thực hiện hai nhiệm vụ song song: chuẩn hóa hình ảnh về không gian đặc trưng của EfficientNet và tổ chức dữ liệu theo thứ tự cố định để phục vụ việc tính toán các chỉ số.

preprocessing_function: Chuẩn hóa ảnh.

x_col / y_col: Xác định cột nào là ảnh (x), cột nào là tên bệnh (y).

target_size: Đồng bộ kích thước, ép mọi ảnh về chuẩn 224 x 224 để khớp với đầu vào mô hình.

batch_size: Nạp 32 ảnh mỗi lần để không gây quá tải bộ nhớ máy tính.

class_mode="categorical": Chuyển tên bệnh thành dạng vector số (One-hot) để máy tính tính toán.

shuffle=False: Không xáo trộn ảnh để đối chiếu chính xác khi vẽ Confusion Matrix.

d. *feature – engineering.py*

```
# Import các hằng số từ preprocessing
try:
    from preprocessing import IMG_SIZE, BATCH_SIZE
except ImportError: # Nếu không tìm thấy file preprocessing, sẽ tự dùng thông số mặc định này
    IMG_SIZE = (224, 224)
    BATCH_SIZE = 32
```

from preprocessing import IMG_SIZE, BATCH_SIZE: Kết nối module, lấy các thông số chuẩn (IMG_SIZE, BATCH_SIZE) đã định nghĩa ở file **preprocessing.py**.

except ImportError: Xử lý khi bị lỗi (có thể là không tìm thấy file) thì sẽ tự lấy kích thước mặc định và số lượng ảnh mỗi lần huấn luyện được định nghĩa trong đây.

```
# Hàm tạo bộ nạp dữ liệu huấn luyện
def get_train_generator(train_df):
    """Tạo generator cho tập train với kỹ thuật Augmentation mạnh"""
    train_datagen = ImageDataGenerator(
        preprocessing_function=preprocess_input, # Chuẩn hóa màu sắc ảnh về dạng mô hình hiểu được
        rotation_range=30, # Xoay ảnh ngẫu nhiên trong khoảng 30 độ
        width_shift_range=0.2, # Dịch chuyển ảnh sang trái/phải 20% chiều rộng
        height_shift_range=0.2, # Dịch chuyển ảnh lên/xuống 20% chiều cao
        shear_range=0.15, # Làm nghiêng ảnh
        zoom_range=0.2, # Phóng to hoặc thu nhỏ ảnh 20%
        horizontal_flip=True, # Lật ngược ảnh theo chiều ngang
        brightness_range=[0.7, 1.3], # Giải quyết lỗi ánh sáng
        fill_mode='nearest' # Xử lý các vùng trống sau khi xoay/dịch ảnh
    )
```

Hàm **get_train_generator** thực hiện kỹ thuật Data Augmentation để tạo ra các biến thể hình ảnh ngẫu nhiên trong quá trình huấn luyện. Mục tiêu cốt lõi của hàm này là nâng cao khả năng tổng quát hóa của mô hình, giúp ngăn chặn hiện tượng quá khớp (Overfitting) và đảm bảo hệ thống có thể nhận diện chính xác bệnh gia cầm trong nhiều điều kiện chụp ảnh thực tế khác nhau.

preprocessing_function=preprocess_input: Chuyển đổi giá trị của các điểm ảnh từ khoảng (0, 255) sang định dạng số học mà mô hình EfficientNet đã được huấn luyện trước đó.

ImageDataGenerator: Công cụ tạo ra các biến thể ảnh mới từ ảnh gốc một cách tự động.

rotation_range=30: Xoay ảnh 30° giúp mô hình nhận diện được bệnh dù góc chụp ảnh bị nghiêng.

width/height_shift: Dịch chuyển ảnh sang trái, phải, lên, xuống. Giúp mô hình nhận biết đối tượng kể cả khi phân gà không nằm giữa khung hình.

shear_range=0.15: Tạo hiệu ứng ảnh chụp từ các góc nhìn chéo, tăng độ linh hoạt.

zoom_range=0.2: Mô phỏng việc chụp ảnh ở khoảng cách gần hoặc xa.

horizontal_flip=True: Lật ngang, tăng gấp đôi dữ liệu bằng cách lật đối xứng ảnh (trái sang phải).

brightness_range: Chỉnh độ sáng, giúp mô hình hoạt động tốt trong cả điều kiện thiếu sáng hoặc quá chói.

fill_mode='nearest': Tự động bù đắp các pixel bị thiếu sau khi xoay hoặc dịch chuyển ảnh.

```
# Trả về bộ nạp ảnh đã được cấu hình
return train_datagen.flow_from_dataframe(
    train_df,
    x_col="filepath", # Cột chứa đường dẫn ảnh trong DataFrame
    y_col="label", # Cột chứa nhãn bệnh
    target_size=IMG_SIZE, # Đưa ảnh về kích thước chuẩn 224x224
    batch_size=BATCH_SIZE, # Nạp ảnh theo từng lô (ví dụ 32 tấm/lần)
    class_mode="categorical", # Phân loại đa lớp (đầu ra là vector xác suất)
    shuffle=True # Xáo trộn thứ tự ảnh sau mỗi vòng lặp để mô hình không overfitting
)
```

train_df: Sử dụng tập dữ liệu 80% đã được chia để huấn luyện.

x_col="filepath": Chỉ định cột chứa đường dẫn vật lý của các tệp tin ảnh trên máy tính.

y_col="label": Chỉ định cột chứa tên loại bệnh để mô hình học đối chiếu.

target_size: Ép ảnh về 224 x 224, đảm bảo số lượng đầu vào đồng nhất cho mạng nơ-ron.

batch_size: Chia nhỏ dữ liệu thành các lô (32 ảnh), giúp máy tính xử lý mượt mà, tránh tràn RAM.

class_mode="categorical": Chuyển nhãn bệnh thành dạng vector số để tính toán xác suất cho 4 loại bệnh.

shuffle=True: Đảo thứ tự ảnh sau mỗi chu kỳ học (Epoch) để mô hình không học thuộc lòng thứ tự ảnh.

e. EfficientNetB0.

```
# Import các module đã tạo
from preprocessing import load_and_split_data, get_val_generator, IMG_SIZE
from feature_engineering import get_train_generator
from preprocessing import SEED
```

from preprocessing import load_and_split_data, get_val_generator, IMG_SIZE: Lấy hàm chia dữ liệu (load_and_split_data), bộ nạp kiểm định (get_val_generator) và kích thước ảnh chuẩn (IMG_SIZE) từ file preprocessing.py.

from feature_engineering import get_train_generator: Lấy hàm `get_train_generator` (chứa các kỹ thuật xoay, lật, phóng to ảnh) từ file `feature_engineering.py`.

```
# Hàm focal_loss giúp mô hình tập trung vào các mẫu khó.
def focal_loss(gamma=2., alpha=.25):
    def focal_loss_fixed(y_true, y_pred):
        pt_1 = tf.where(tf.equal(y_true, 1), y_pred, tf.ones_like(y_pred)) # Xác suất dự đoán đúng cho lớp thực tế
        return -tf.reduce_mean(alpha * tf.pow(1. - pt_1, gamma) * tf.math.log(pt_1)) # Giảm trọng số của các mẫu dễ học
    return focal_loss_fixed
```

Focal Loss để tối ưu hóa quá trình huấn luyện. Kỹ thuật này đặc biệt hiệu quả trong việc xử lý sự mất cân bằng giữa các lớp bệnh và các mẫu dữ liệu có độ khó khác nhau. Bằng cách bổ sung hệ số điều chỉnh $(1 - p_i)^\gamma$, hàm loss sẽ giảm thiểu tác động của các mẫu dễ nhận diện và tập trung điều chỉnh trọng số dựa trên các mẫu khó phân loại. Điều này giúp mô hình đạt được độ chính xác cao hơn trên các ca bệnh phức tạp, vốn thường bị bỏ qua.

```
# Xây dựng kiến trúc mô hình
def build_model(num_classes):
    base_model = EfficientNetB0( # Sử dụng mô hình EfficientNetB0 đã được huấn luyện sẵn trên tập ImageNet
        weights="imagenet",
        include_top=False,
        input_shape=(*IMG_SIZE, 3)
    )
    base_model.trainable = False # Đóng băng mô hình gốc để không làm hỏng kiến trúc cũ trong giai đoạn đầu
```

weights="imagenet": Học chuyển đổi (Transfer Learning) sử dụng kiến thức có sẵn từ mô hình đã học trên 1.4 triệu ảnh (ImageNet). Điều này giúp mô hình nhận diện được các đường nét, màu sắc cơ bản ngay lập tức mà không cần học lại từ đầu.

include_top=False: Cắt bỏ phần đầu (lớp đầu ra) của ImageNet (vốn dùng để phân loại 1000 vật thể) để thay thế bằng các lớp mới chuyên nhận diện các loại bệnh gia cầm.

input_shape=(*IMG_SIZE, 3): Thiết lập khung nhận diện ảnh theo kích thước chuẩn 224 x 224 và 3 kênh màu (RGB).

base_model.trainable = False: Đóng băng giữ nguyên các trọng số đã học được của EfficientNetB0. Ở giai đoạn đầu, chỉ huấn luyện phần đầu mới thêm vào, giúp quá trình học ổn định và nhanh chóng hơn.

```
inputs = layers.Input(shape=(*IMG_SIZE, 3))
x = base_model(inputs, training=False)
x = layers.GlobalAveragePooling2D()(x) # Chuyển đổi dữ liệu từ không gian 2D sang vector phẳng
x = layers.BatchNormalization()(x) # Chuẩn hóa dữ liệu giúp mô hình hội tụ nhanh hơn

x = layers.Dense(512, activation='relu')(x) # Thêm các lớp Dense để học các đặc trưng riêng của phân gà
x = layers.BatchNormalization()(x)
x = layers.Dropout(0.4)(x) # Ngẫu nhiên ngắt kết nối 40% nơ-ron để chống Overfitting

x = layers.Dense(256, activation='relu')(x)
x = layers.BatchNormalization()(x)
x = layers.Dropout(0.3)(x) # Ngẫu nhiên ngắt kết nối 30% nơ-ron để chống Overfitting

outputs = layers.Dense(num_classes, activation="softmax")(x) # Lớp đầu ra với hàm Softmax để xuất xác suất cho từng loại bệnh
return models.Model(inputs, outputs), base_model
```

GlobalAveragePooling2D: Chuyển đổi các bản đồ đặc trưng phức tạp thành một vector phẳng duy nhất, giúp giảm số lượng tham số và tránh Overfitting.

BatchNormalization: Chuẩn hóa dữ liệu sau mỗi lớp, giúp mô hình học nhanh hơn và giảm sự phụ thuộc vào việc khởi tạo trọng số ban đầu.

Dense (512, 256): Các lớp kết nối đầy đủ giúp mô hình tổng hợp các đặc trưng đã học được để nhận diện các dấu hiệu bệnh cụ thể.

Dropout (0.4, 0.3): Ngắt kết nối ngẫu nhiên một tỷ lệ nơ-ron trong lúc học, buộc mô hình phải học những đặc trưng tổng quát nhất.

activation="softmax": Chuyển đổi kết quả cuối cùng thành dạng xác suất.

```
# Chạy huấn luyện
if __name__ == "__main__":
    # Load Data
    train_df, val_df = load_and_split_data()
    train_gen = get_train_generator(train_df)
    val_gen = get_val_generator(val_df)

    # Khởi tạo model
    model, base_model = build_model(num_classes=len(train_gen.class_indices))
```

if __name__ == "__main__": Đảm bảo toàn bộ quy trình huấn luyện chỉ bắt đầu khi trực tiếp chạy file này.

build_model(): Xây dựng kiến trúc mạng EfficientNetB0 với số lượng lớp đầu ra khớp với số lượng bệnh thực tế có trong dữ liệu.

```
# Tính Class Weights, xử lý mất cân bằng dữ liệu
weights = class_weight.compute_class_weight('balanced', classes=np.unique(train_gen.classes), y=train_gen.classes)
cw_dict = dict(enumerate(weights))
cw_dict[3] *= 0.8 # Giảm Salmonella
cw_dict[2] *= 1.2 # Tăng Newcastle
```

Để giảm bớt tình trạng mất cân bằng dữ liệu do lớp Newcastle có quá ít dữ liệu cần sử dụng kỹ thuật Class weights. Thiết lập trọng số cao cho Newcastle để mô hình chú ý vào lớp này nhiều hơn và giảm nhẹ lớp salmonella để tránh bị nhầm lẫn với các lớp khác.

compute_class_weight(): Thư viện Sklearn sẽ tính toán để đưa ra trọng số cao cho lớp ít ảnh và trọng số thấp cho lớp nhiều ảnh.

cw_dict = dict(enumerate(weights)): Chuyển danh sách trọng số thành một từ điển để mô hình AI dễ dàng tra cứu khi học.

cw_dict[3] *= 0.8: Can thiệp thủ công (giảm) nhận thấy lớp Salmonella đang dễ bị nhận nhầm hoặc có quá nhiều mẫu, nên giảm tầm quan trọng của nó xuống một chút.

cw_dict[2] *= 1.2: Can thiệp thủ công (tăng) vì lớp Newcastle có quá ít dữ liệu nên phải đặc biệt chú ý đến lớp này.

```
# Giai đoạn 1
callbacks_fe = [
    tf.keras.callbacks.ModelCheckpoint("EfficienNetB0.keras", monitor="val_loss", save_best_only=True, verbose=1), # Tự động lưu lại mô |
    tf.keras.callbacks.EarlyStopping(monitor="val_loss", patience=7, restore_best_weights=True, verbose=1) # Tự động dừng huấn luyện sớm
]
```

ModelCheckpoint: Trong suốt quá trình học (ví dụ 20 vòng), mô hình sẽ tự động lưu lại file EfficienNetB0.keras mỗi khi nó thấy độ chính xác đạt mức cao mới.

monitor="val_loss": Lấy độ lỗi trên tập kiểm định làm thước đo. Loss càng thấp thì mô hình càng giỏi.

save_best_only=True: Chỉ lưu bản tốt nhất.

EarlyStopping: Nếu sau một số vòng lặp nhất định (patience=7) mà mô hình không tiến bộ thêm chút nào, chương trình sẽ tự ngắt huấn luyện.

patience=7: Cho phép mô hình "thử sai" thêm 7 lần. Nếu sau 7 lần vẫn không khá hơn thì mới dừng hẳn.

restore_best_weights: Khi dừng sớm, nó sẽ tự động lấy lại những thông số của vòng lặp tốt nhất trước đó để sử dụng.

```
# Cấu hình mô hình
model.compile(
    optimizer=tf.keras.optimizers.Adam(1e-3),
    loss=tf.keras.losses.CategoricalCrossentropy(label_smoothing=0.05), # Label smoothing giúp mô hình bớt tự tin
    metrics=["accuracy"]
)
```

Adam(1e-3): giúp mô hình cập nhật trọng số một cách thông minh và nhanh chóng ở giai đoạn đầu.

CategoricalCrossentropy: Đo lường sự khác biệt giữa dự đoán của máy và thực tế để điều chỉnh mô hình.

metrics=["accuracy"]: Dùng để theo dõi tỉ lệ đoán đúng của mô hình.

```
# Bắt đầu huấn luyện
history_fe = model.fit(
    train_gen,
    validation_data=val_gen,
    epochs=5,
    callbacks=callbacks_fe
)
```

Huấn luyện giai đoạn 1 (trích xuất đặt trung).

Quá trình huấn luyện được khởi tạo thông qua hàm model.fit với sự phối hợp giữa tập dữ liệu huấn luyện (train_gen) và tập kiểm định (validation_data). Trong giai đoạn này, mô

hình thực hiện 5 chu kỳ lặp (epochs) để tối ưu hóa các lớp phân loại mới được thêm vào. Toàn bộ diễn biến về độ chính xác và giá trị mất mát được ghi lại trong biến `history_fe`.

```
# Giai đoạn 2
# Fine-tuning
base_model.trainable = True # Mở đóng băng toàn bộ
for layer in base_model.layers[:-60]: # đóng băng lại trừ 60 lớp cuối cùng
    layer.trainable = False
```

base_model.trainable = True: Mở khóa toàn bộ cho phép tất cả các lớp của mô hình EfficientNetB0 có thể cập nhật trọng số.

base_model.layers[:-60]: layer.trainable = False: Đóng băng các lớp lại chỉ mở 60 lớp cuối cùng.

```
callbacks_ft = [
    tf.keras.callbacks.ModelCheckpoint("EfficientNetB0.keras", monitor="val_loss", save_best_only=True, verbose=1), # Tự động lưu lại mô
    tf.keras.callbacks.EarlyStopping(monitor="val_loss", patience=7, restore_best_weights=True, verbose=1), # Tự động dừng huấn luyện sớm
    tf.keras.callbacks.ReduceLROnPlateau(monitor="val_loss", factor=0.5, patience=3, verbose=1) # Tự động giảm Learning Rate nếu mô hình
]
```

Callbacks cho giai đoạn hai.

ReduceLROnPlateau: Khi mô hình học đến một ngưỡng mà độ lỗi (`val_loss`) không giảm thêm được nữa, nó sẽ tự động "phanh" lại (giảm tốc độ học) để dò tìm điểm tối ưu kỹ hơn.

factor=0.5: Hệ số giảm khi kích hoạt, tốc độ học sẽ bị giảm đi một nửa.

patience=3: Nếu sau 3 vòng lặp (epoch) mà kết quả không tốt lên, nó sẽ thực hiện giảm tốc độ học ngay lập tức.

```
#Cấu hình mô hình giai đoạn 2
model.compile(
    optimizer=tf.keras.optimizers.Adam(1e-5),
    loss=focal_loss(),
    metrics=["accuracy"]
)
```

Cấu hình mô hình cho giai đoạn 2.

Adam(1e-5): Giảm tốc độ học giúp mô hình cập nhật các trọng số một cách cực kỳ cẩn thận, chỉ thay đổi rất nhỏ để phù hợp với ảnh phân gà mà không làm mất đi khả năng nhận diện hình ảnh căn bản.

focal_loss(): giúp mô hình tập trung tối đa vào các mẫu ảnh khó phân loại trong giai đoạn tinh chỉnh cuối cùng.

```
# Bắt đầu huấn luyện giai đoạn 2
history_ft = model.fit(
    train_gen,
    validation_data=val_gen,
    epochs=15,
    class_weight=cw_dict,
    callbacks=callbacks_ft
)
```

Giai đoạn huấn luyện 2 (fine – tuning).

Giai đoạn huấn luyện thứ hai (Fine-tuning) được triển khai với 15 chu kỳ lặp nhằm tối ưu hóa sâu các tầng đặc trưng của mạng nơ-ron. Sử dụng kỹ thuật class weights để xử lý việc dữ liệu bị mất cân bằng và focal loss để xử lý các mẫu khó dễ bị nhầm sang lớp khác. Kết hợp lại giúp mô hình đạt trạng thái hội tụ tốt nhất. Kết quả sẽ được cải thiện rõ rệt so với kết quả đạt được qua lần huấn luyện đầu tiên.

f. evaluation.py

```
def plot_training_history(history_fe, history_ft):
    """Hàm nối dữ liệu và vẽ biểu đồ Accuracy/Loss từ 2 giai đoạn"""
    # Hàm nối dữ liệu history nội bộ
    def merge_histories(h1, h2):
        out = {}
        for k in h1.history.keys():
            out[k] = h1.history[k] + h2.history.get(k, []) # Nối danh sách các chỉ số (accuracy, loss) của 2 giai đoạn lại làm một
        return out

    hist = merge_histories(history_fe, history_ft)
    len_fe = len(history_fe.history["accuracy"]) # Điểm đánh dấu kết thúc giai đoạn 1
```

def plot_training_history(history_fe, history_ft): Định nghĩa hàm nhận vào dữ liệu của hai giai đoạn huấn luyện.

def merge_histories(h1, h2): Đây là một hàm phụ nằm bên trong để thực hiện việc nối danh sách các chỉ số. `out = {}` khởi tạo một từ điển (dictionary) trống để chứa dữ liệu sau khi nối.

for k in h1.history.keys(): Duyệt qua từng loại chỉ số trong dữ liệu huấn luyện ví dụ: `loss`, `accuracy`, `val_loss`, `val_accuracy`.

out[k] = h1.history[k] + h2.history.get(k, []): Lấy danh sách giá trị của giai đoạn 1 cộng với danh sách giá trị của giai đoạn 2. Kết quả là một mảng dài liên tục từ đầu đến cuối.

hist = merge_histories(history_fe, history_ft): Gọi hàm vừa định nghĩa ở trên để tạo ra bộ dữ liệu tổng hợp cuối cùng lưu vào biến `hist`.

len_fe = len(history_fe.history["accuracy"]): Đếm xem Giai đoạn 1 đã chạy bao nhiêu vòng (epoch) để có thể đánh dấu bắt đầu qua giai đoạn 2.

```

# Biểu đồ Accuracy
plt.subplot(1, 2, 1)
plt.plot(hist["accuracy"], label="Train Accuracy", marker='o', markersize=3)
plt.plot(hist["val_accuracy"], label="Val Accuracy", marker='o', markersize=3)
# Vẽ đường kẻ đỏ ngăn cách giữa giai đoạn 1 và 2
plt.axvline(x=len_fe - 1, color='red', linestyle='--', label='Start Fine-tuning')
plt.title("Model Accuracy (EfficientNetB0)")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend(loc="lower right")
plt.grid(True, alpha=0.3)

```

Vẽ biểu đồ độ chính xác qua các epochs.

Biểu đồ được thiết kế với các điểm đánh dấu (markers) tại mỗi chu kỳ để theo dõi sát sao tiến trình hội tụ. Đặc biệt, một đường kẻ dọc màu đỏ nét đứt được tích hợp tại điểm `len_fe - 1` để phân tách rõ rệt giữa giai đoạn trích xuất đặc trưng và giai đoạn tinh chỉnh sâu.

```

# Biểu đồ Loss
plt.subplot(1, 2, 2)
plt.plot(hist["loss"], label="Train Loss", marker='o', markersize=3)
plt.plot(hist["val_loss"], label="Val Loss", marker='o', markersize=3)
# Vẽ đường kẻ đỏ ngăn cách giữa giai đoạn 1 và 2
plt.axvline(x=len_fe - 1, color='red', linestyle='--', label='Start Fine-tuning')
plt.title("Model Loss (Focal Loss)")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend(loc="upper right")
plt.grid(True, alpha=0.3)

```

Biểu đồ hàm mất mát (Loss) qua các epochs.

```

def evaluate_with_threshold(model, val_gen, threshold_value=0.5, s_idx=3):
    """Hàm dự đoán với kỹ thuật Threshold Tuning"""
    val_gen.reset() # Đặt lại bộ nạp ảnh về vị trí đầu tiên
    y_pred_probs = model.predict(val_gen, verbose=1) # Dự đoán xác suất cho toàn bộ tập Validation

    y_pred_tuned = []
    # Duyệt qua từng kết quả dự đoán (dạng xác suất)
    for prob in y_pred_probs:
        # Nếu xác suất Salmonella không vượt qua ngưỡng tin cậy mới
        if prob[s_idx] < threshold_value:
            p_temp = prob.copy()
            p_temp[s_idx] = 0 # Ép lớp Salmonella về 0
            y_pred_tuned.append(np.argmax(p_temp)) # Chọn lớp có xác suất cao thứ nhì
        else:
            y_pred_tuned.append(np.argmax(prob)) # Nếu vượt ngưỡng thì tin tưởng kết quả mô hình

    y_pred_tuned = np.array(y_pred_tuned)
    y_true = val_gen.classes # Nhân thực tế từ tập dữ liệu

```

Threshold Tuning: một kỹ thuật tối ưu hóa sau huấn luyện nhằm tinh chỉnh khả năng ra quyết định của mô hình. Trong bài toán này, thay vì sử dụng phương pháp lấy nhãn có xác suất cao nhất một cách máy móc, ta thiết lập một ngưỡng tin cậy chủ động cho các lớp có nguy cơ nhầm lẫn cao. Cụ thể, kết quả chỉ được công nhận là bệnh Salmonella khi độ tự tin của mô hình vượt mức 50%. Nếu không đạt ngưỡng, hệ thống sẽ tự động cân nhắc các phương án dự phòng có xác suất cao kế tiếp. Điều này giúp cân bằng giữa độ chính xác (Precision) và độ triệu hồi (Recall), đồng thời tăng cường tính thực tiễn của mô hình trong môi trường chẩn đoán lâm sàng.

val_gen.reset(): Đưa con trỏ của bộ nạp ảnh về vị trí đầu tiên.

model.predict(): Lấy ra độ tin cậy của mô hình đối với tất cả các lớp bệnh thay vì chỉ lấy lớp cao nhất.

if prob[s_idx] < threshold_value: Kiểm tra xem xác suất của lớp Salmonella có đủ lớn không.

p_temp[s_idx] = 0: Nếu không chắc chắn, ta ép mô hình phải cân nhắc các khả năng khác.

np.argmax(p_temp): Chọn phương án dự phòng an toàn nhất khi phương án ưu tiên không đủ độ tin cậy.

y_true = val_gen.classes: Dùng làm "đáp án đúng" để so sánh với kết quả sau khi đã điều chỉnh ngưỡng.

```
# Tính toán và vẽ Confusion Matrix
cm = confusion_matrix(y_true, y_pred_tuned)
plt.figure(figsize=(10, 8))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
            xticklabels=class_names, yticklabels=class_names)
plt.title(f'Confusion Matrix (Salmonella Threshold: {threshold_value})', fontsize=16)
plt.ylabel('Actual Label', fontsize=12)
plt.xlabel('Predicted Label', fontsize=12)
plt.show()
```

Mã trận nhầm lẫn được xây dựng để cung cấp cái nhìn chi tiết về hiệu năng dự đoán của mô hình trên từng lớp bệnh cụ thể sau khi áp dụng kỹ thuật Threshold Tuning.

3. Mô hình CNN

- Tăng cường dữ liệu:


```

# =====
# CNN
# =====
def get_train_val_generators_cnn(train_df, val_split=0.2):
    # 3. Tiền xử lý & Tăng cường dữ liệu (Augmentation)
    # ImageDataGenerator trong Keras dùng để chuẩn bị và biến đổi dữ liệu ảnh
    # Chuẩn hóa dữ liệu (Normalization) → giúp mô hình học nhanh và ổn định
    # Tăng cường dữ liệu (Data Augmentation) → tạo thêm biến thể mới của ảnh
    train_datagen = ImageDataGenerator(
        rescale=1./255,          # Chuẩn hóa giá trị pixel về [0, 1]
        rotation_range=20,       # Xoay nhẹ ảnh trong khoảng ±20 độ --> Giảm nhiễu
        width_shift_range=0.2,    # Dịch ngang lên đến 20% chiều rộng ảnh -
        height_shift_range=0.2,   # Dịch dọc lên đến 20% chiều cao ảnh --> Giảm nhiễu
        zoom_range=0.2,          # Phóng to/thu nhỏ ảnh trong khoảng ±20%
        horizontal_flip=True,     # Lật ngang --> Hữu ích cho ảnh đối xứng
        validation_split=val_split # Tách tập validation

    )

    # Tạo generator cho tập huấn luyện
    train_gen = train_datagen.flow_from_dataframe(
        train_df,
        x_col="filepath",
        y_col="label",
        target_size=IMG_SIZE_CNN,
        batch_size=BATCH_SIZE,
        subset='training',
        class_mode='categorical'
    )

    # Tạo generator cho tập validation
    val_gen = train_datagen.flow_from_dataframe(
        train_df,
        x_col="filepath",
        y_col="label",
        target_size=IMG_SIZE_CNN,
        batch_size=BATCH_SIZE,
        subset='validation',
        class_mode='categorical',
        shuffle=False
    )

    return train_gen, val_gen

```

- Build model

```

def build_cnn_model(num_classes):
    # 4. Xây dựng mô hình CNN
    # Đây là một mạng CNN đơn giản dùng để phân loại ảnh
    # Mỗi lớp (layer) trong mô hình có một vai trò riêng
    model = models.Sequential([
        layers.Input(shape=(150,150,3)), #Xác định kích th
        layers.Conv2D(32, (3,3), activation='relu'), #32:
        #(3,3): kích thước kernel (bộ lọc) 3x3 dùng để qu
        #-->trích xuất đặc trưng cơ bản như cạnh, góc, màu
        layers.MaxPooling2D(2,2), #(2,2) có nghĩa là lấy g

        layers.Conv2D(64, (3,3), activation='relu'), #Lớp
        layers.MaxPooling2D(2,2),

        layers.Conv2D(128, (3,3), activation='relu'), #Lớp
        layers.MaxPooling2D(2,2),

        layers.Flatten(), #Biến đầu ra 3D (chiều cao x chi
        layers.Dense(128, activation='relu'), #Lớp Fully C
        layers.Dropout(0.3), # Dropout ngẫu nhiê
        layers.Dense(num_classes, activation='softmax') #
        # mô hình chọn lớp có xác suất cao nhất.
    ])
    return model

```

- Train model:


```

def train_cnn(EPOCHS=5):
    tf.random.set_seed(SEED) #cố định tính ngẫu nhiên (random) của TensorFlow (l
    np.random.seed(SEED) #Cố định random của NumPy (Chia train/va, Tạo số ngẫu n

    train_df, val_df = load_and_split_data()

    train_gen, val_gen = get_train_val_generators_cnn(train_df, val_split=0.2)

    # Lấy tên các lớp
    NUM_CLASSES = len(train_gen.class_indices)
    print("Classes:", train_gen.class_indices)
    print("NUM_CLASSES:", NUM_CLASSES)

    model = build_cnn_model(NUM_CLASSES)
    model.summary()

    # 5. Compile mô hình
    model.compile(
        optimizer='adam',
        loss='categorical_crossentropy',
        metrics=['accuracy']
    )

    # 6.Huấn luyện mô hình
    history = model.fit(
        train_gen,
        validation_data=val_gen,
        epochs=EPOCHS
    )

    os.makedirs("models", exist_ok=True)
    os.makedirs("results/history", exist_ok=True)

    save_history(history, "results/history/cnn_history.json")

    #lưu TOÀN BỘ mô hình vào file và Load lại mô hình đã lưu
    model.save("models/CNN_final.keras")

    return model, history, val_gen

```

- Chạy độc lập

```
✓ if __name__ == "__main__":  
    # chạy: python -m src.model_cnn  
    model, history, val_gen = train_cnn(EPOCHS=5)  
  
    from .evaluation import plot_history_from_json, evaluate_confusion_report  
    import json  
  
    with open("results/history/cnn_history.json", "r", encoding="utf-8") as f:  
        h = json.load(f)  
  
    plot_history_from_json(h, None, title_prefix="CNN",  
                           save_path="results/figures/cnn_history.png")  
    evaluate_confusion_report(model, val_gen, save_dir="results/figures", prefix="cnn")
```

- Evaluation

```
elif model_name == 'cnn':
    model_path = "models/CNN_final.keras"
    hist_path = "results/history/cnn_history.json"

    from .preprocessing import load_and_split_data
    from .feature import get_train_val_generators_cnn

    train_df, val_df = load_and_split_data()
    # CNN split ngay trong generator train_datagen
    train_gen, val_gen = get_train_val_generators_cnn(train_df, val_split=0.2)

    loaded = tf.keras.models.load_model(model_path, compile=False)

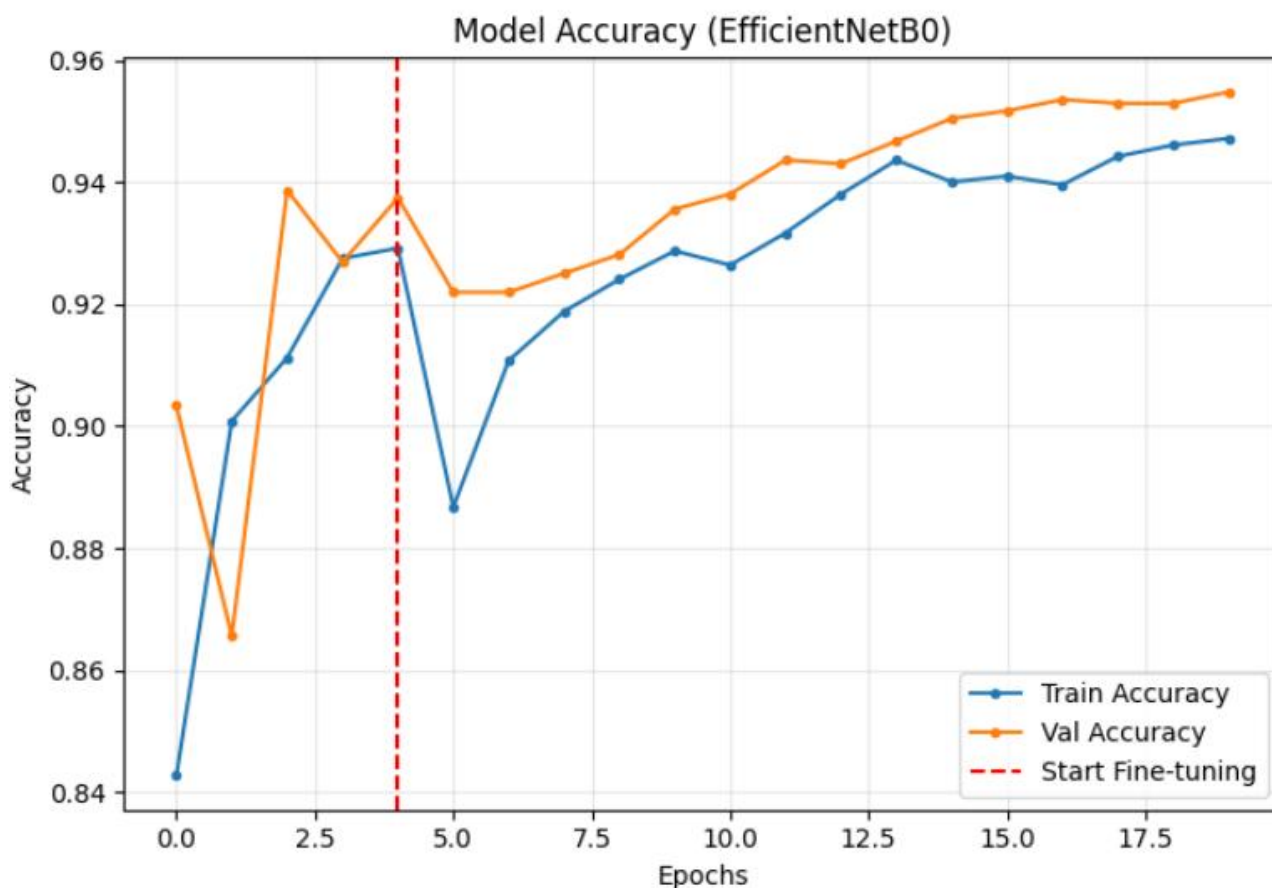
    h = _safe_load_history(hist_path)
    plot_history_from_json(h, None, title_prefix="CNN",
                          save_path=os.path.join(save_dir, "cnn_history.png"))

    evaluate_confusion_report(loaded, val_gen, save_dir=save_dir, prefix="cnn")
```

Chương 4: Đánh giá mô hình.

1. EfficientNetB0.

1.1 Biểu đồ Accuracy.



a. Giai đoạn huấn luyện 1(5 epochs).

Tại epochs 1 chỉ số train accuracy đạt 84,27%, val accuracy đạt 90,33%. Sau đó tại epochs cuối của giai đoạn huấn luyện 1 (epochs 5). Hai chỉ số này tăng lên đáng kể: Train accuracy tăng lên 92,92%, tăng 8,65% và val accuracy tăng lên 93,74%, tăng 3,41%.

Nhận xét: Mô hình có khởi đầu khá tốt với chỉ số accuracy ở cả hai tập train và val khá cao (hơn 80%). Ở tập val accuracy khi đến epochs 2 thì có sụt giảm mạnh (giảm khoảng 4%) nhưng sau đó đã tăng lại lên ở epoch. Mô hình đã nhanh chóng tự điều chỉnh và duy trì xu hướng hội tụ ổn định.

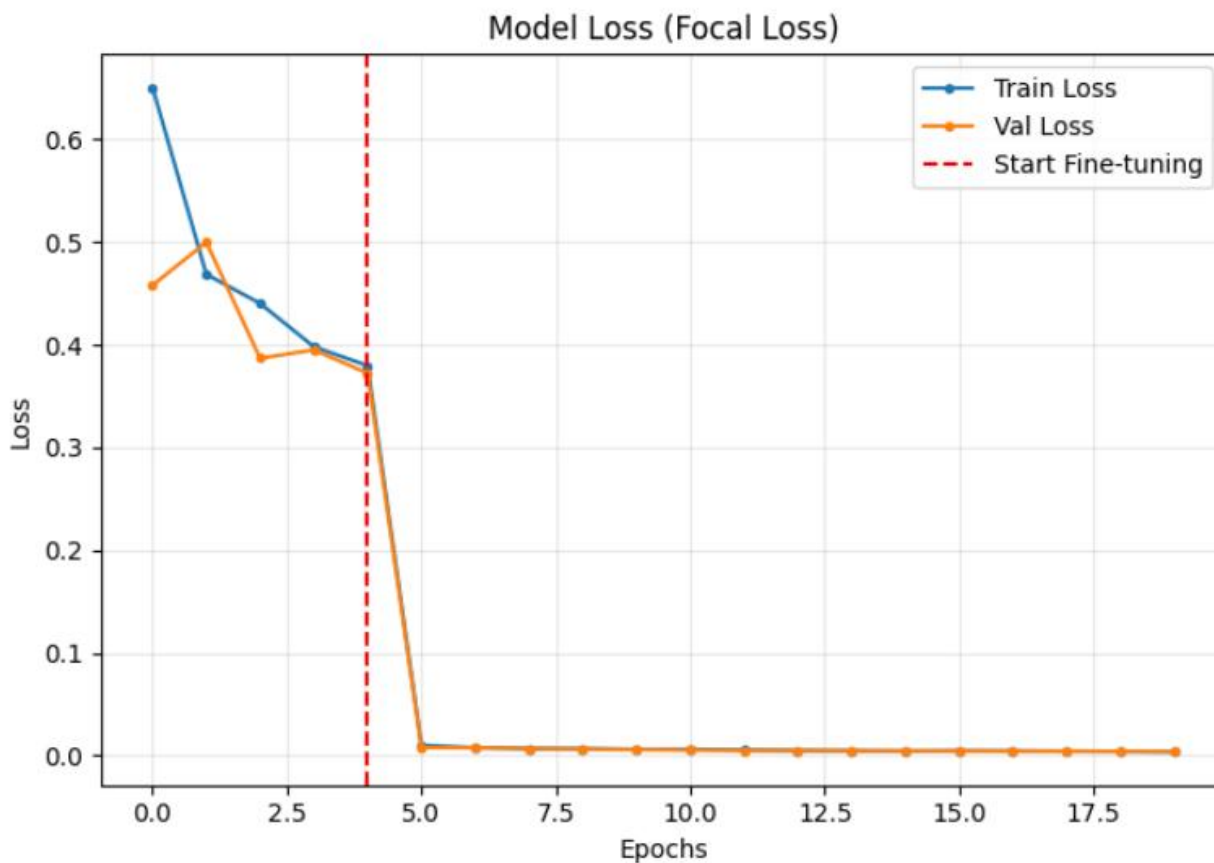
Các chỉ số của tập val cao hơn so với tập train cho thấy mô hình không bị overfitting. Nhận thấy mô hình đã học được các đặc trưng cơ bản cần thiết.

b. Giai đoạn huấn luyện 2 (15 epochs).

Sau khi fine-tuning mở 60 lớp cuối và đóng băng các lớp còn lại kết hợp với focal_loss và class_weight. Ở ngay epochs 1 của giai đoạn 2, hai chỉ có train acc bị sụt giảm còn 88,67% và val acc 92,19% nhưng đến epochs 2 thì train acc tăng lên 91,07% còn val acc vẫn tăng không đáng kể. Nguyên nhân có thể do mở 60 lớp cuối khiến cho mô hình sốc, mô hình phải tính toán lại các giá trị khi có thêm 60 lớp này. Đến epochs 15 thì train acc tăng lên 94,72% và val acc tăng 95,48%.

Nhận xét: Giai đoạn 2 (Fine-tuning) đã chứng minh hiệu quả vượt trội trong việc tối ưu hóa mô hình. Với tốc độ học ổn định ở mức 10^{-5} mô hình không chỉ bảo toàn được các đặc trưng đã học mà còn tinh chỉnh sâu các lớp nơ-ron để đạt độ chính xác kiểm định tối ưu là 95.48%. Biểu đồ cho thấy xu hướng hội tụ sau vạch đỏ diễn ra rất ổn định và mượt mà, không xuất hiện hiện tượng dao động mạnh hay quá khớp (Overfitting).

1.2 Biểu đồ Loss.



a. Giai đoạn 1(5 epochs).

Ở ngay epochs 1 train loss đạt 0,65 và val loss 0,4581 epochs 5 thì train loss đã giảm xuống còn 0,3798 và val loss còn 0,3724.

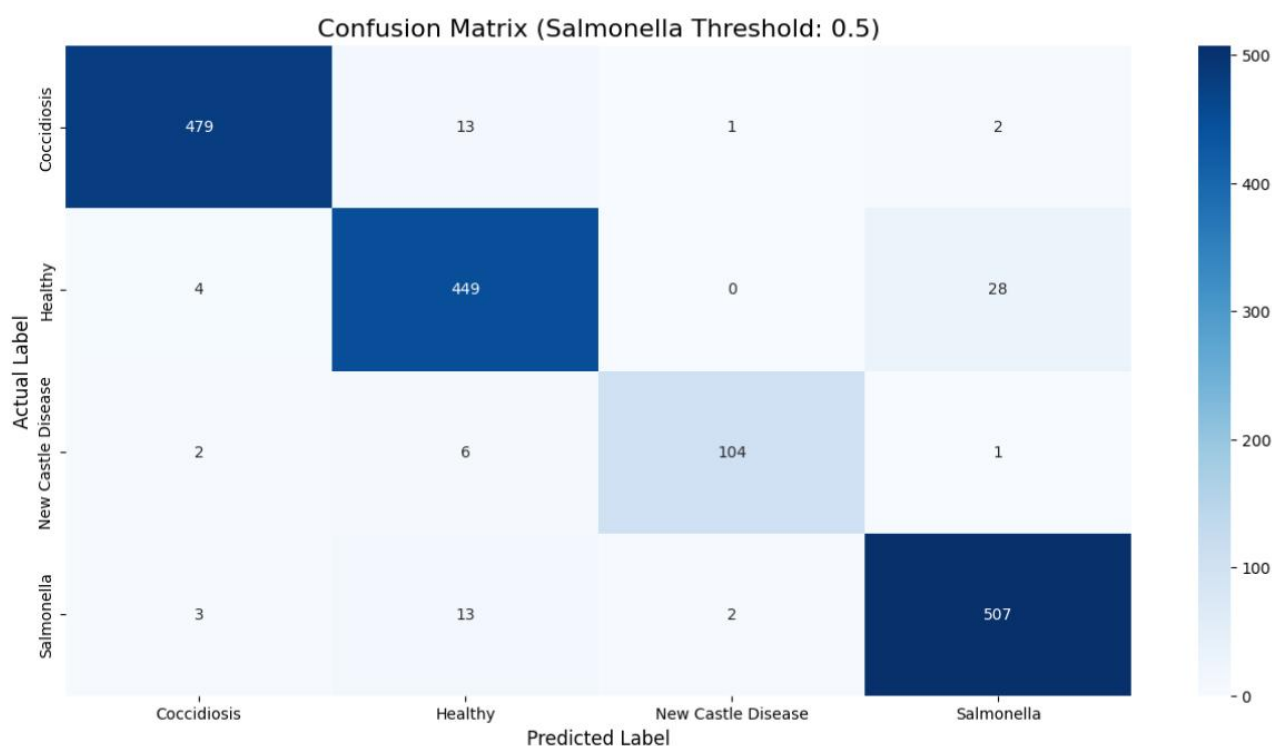
Nhận xét: biểu đồ hàm mất mát Loss thể hiện tiến trình hội tụ ổn định với xu hướng giảm từ 0,65 xuống 0,3798. Nhìn vào biểu đồ có thể thấy Val Loss không bị tách xa khỏi Train Loss (thậm chí đôi khi thấp hơn) khẳng định rằng ở giai đoạn này, mô hình hoàn toàn không bị học vẹt (Overfitting). Mặc dù train loss và val loss vẫn còn ở mức cao do các lớp của mô hình vẫn đang bị đóng băng mô hình chỉ có thể tối ưu lớp phân loại cuối cùng. Vì vậy, sai số giảm chậm vì năng lực học của mô hình bị giới hạn ở các lớp sâu.

b. Giai đoạn 2(15 epochs).

Ngay tại epochs 1 có thể thấy sự sụt giảm mạnh của train loss và val loss, train loss chỉ còn 0,0100 và val loss còn 0,0078. Epochs 15 thì train loss còn 0,0036 và val loss 0,0039.

Nhận xét: Ngay sau vạch đỏ (điểm bắt đầu Fine-tuning), giá trị Loss ghi nhận một cú sụt giảm gần như "thẳng đứng" từ mức 0.3798 xuống mức 0.0039. Điều này chứng tỏ việc mở khóa 60 lớp cuối của mạng xương sống EfficientNetB0 đã giải phóng "năng lực học" của mô hình, cho phép hệ thống can thiệp và tối ưu hóa các trọng số ở tầng sâu. Đường train loss và val loss gần như trùng khít lên nhau và đi ngang ở đoạn cuối biểu đồ. Đây là trạng thái lý tưởng trong huấn luyện mô hình học sâu, minh chứng cho việc mô hình đã đạt đến điểm hội tụ tối ưu, không có dấu hiệu bị dao động hay quá khớp (Overfitting) dù đã được mở khóa nhiều tham số hơn.

1.3 Ma trận nhầm lẫn(confusion matrix).



- Coccidiosis: dự đoán đúng 479 mẫu, 13 mẫu bị nhầm sang Healthy, các lớp khác rất ít.
- Healthy: dự đoán đúng 449 mẫu, có 28 mẫu bị nhầm sang Salmonella, các lớp còn lại rất ít.
- New Caslte Disease: dự đoán đúng 104 mẫu, dự đoán nhầm rất ít.
- Salmonella: dự đoán đúng 507 mẫu, có 13 mẫu bị nhầm sang Healthy, các lớp còn lại rất ít.

Nhận xét: Ma trận nhầm lẫn cung cấp một cái nhìn toàn diện về hiệu năng của mô hình trên cả 4 lớp dữ liệu. Đường chéo chính của ma trận có màu rất đậm cho thấy mô hình hoạt động rất hiệu quả. Vẫn có một số lỗi dự đoán nhầm nhưng tỷ lệ cực kỳ thấp và không gây ảnh hưởng đến khả năng dự đoán của mô hình. Có thể thấy mô hình này có độ tin cậy cao, khả năng phân loại các đặc trưng tốt và cực kỳ ổn định trong việc phân loại các loại bệnh ở gà.

1.4 Các chỉ số.

	precision	recall	f1-score	support
Coccidiosis	0.98	0.97	0.97	495
Healthy	0.93	0.93	0.93	481
New Castle Disease	0.97	0.92	0.95	113
Salmonella	0.94	0.97	0.95	525
accuracy			0.95	1614
macro avg	0.96	0.95	0.95	1614
weighted avg	0.95	0.95	0.95	1614

Coccidiosis:

- Precision: đạt 0,98 cao nhất trong các lớp. Khi mô hình dự đoán là Coccidiosis, xác suất đúng lên tới 98%. Hầu như không có sự nhầm lẫn từ các bệnh khác sang bệnh này.
- Recall: đạt 0,97. Mô hình nhận diện được 97% các mẫu thực tế bị Coccidiosis.
- F1 – score: cao nhất trong các lớp 0.97. Mô hình gần như đạt đến độ hoàn hảo trong việc dự đoán bệnh Cocci.

Healthy:

- Precision: đạt 0,93.
- Recall: đạt 0,93.

- F1 – score: đạt 0,93.
- ⇒ Phân loại gà khỏe mạnh có độ tin cậy cao, tránh việc dùng thuốc lãng phí.

New Castle Disease:

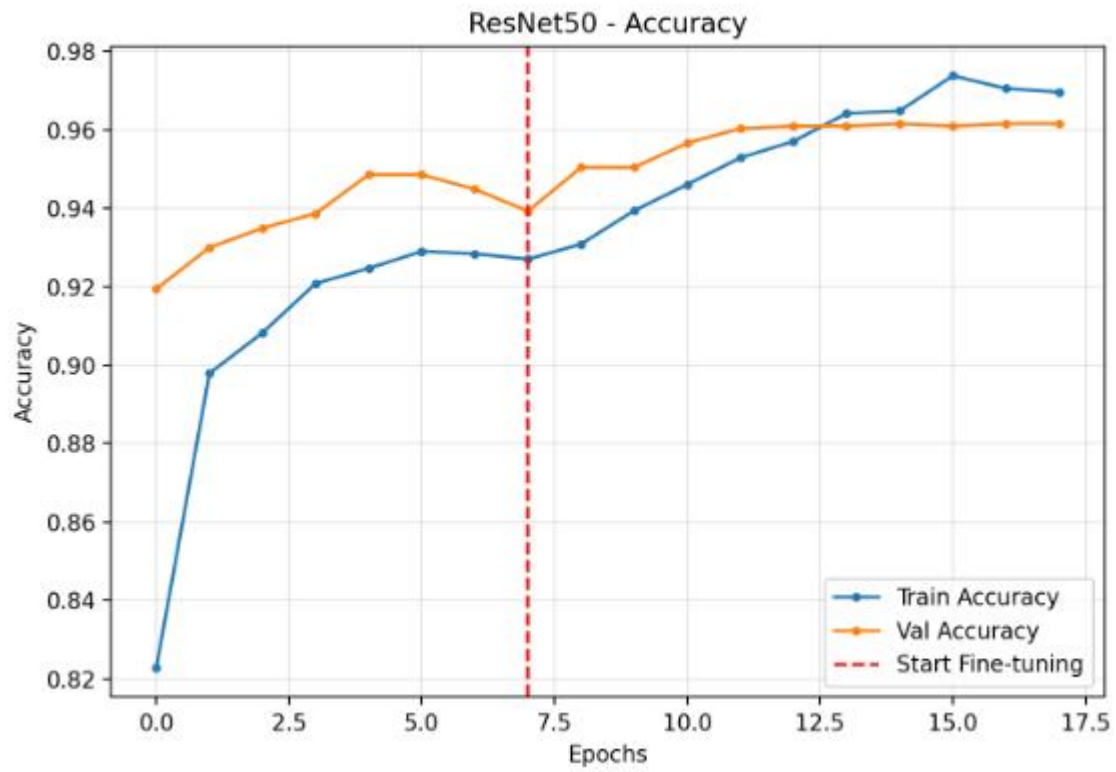
- Precision: đạt 0,97. Độ chính xác rất cao, ít khi đoán nhầm các bệnh khác thành New Castle.
- Recall: đạt 0,92 thấp nhất trong các lớp. Có thể do lớp này có ít dữ liệu.
- F1 - score: đạt 0,95. Cho thấy mô hình vẫn cực kỳ ổn định và không bị ảnh hưởng quá nhiều bởi sự mất cân bằng dữ liệu.

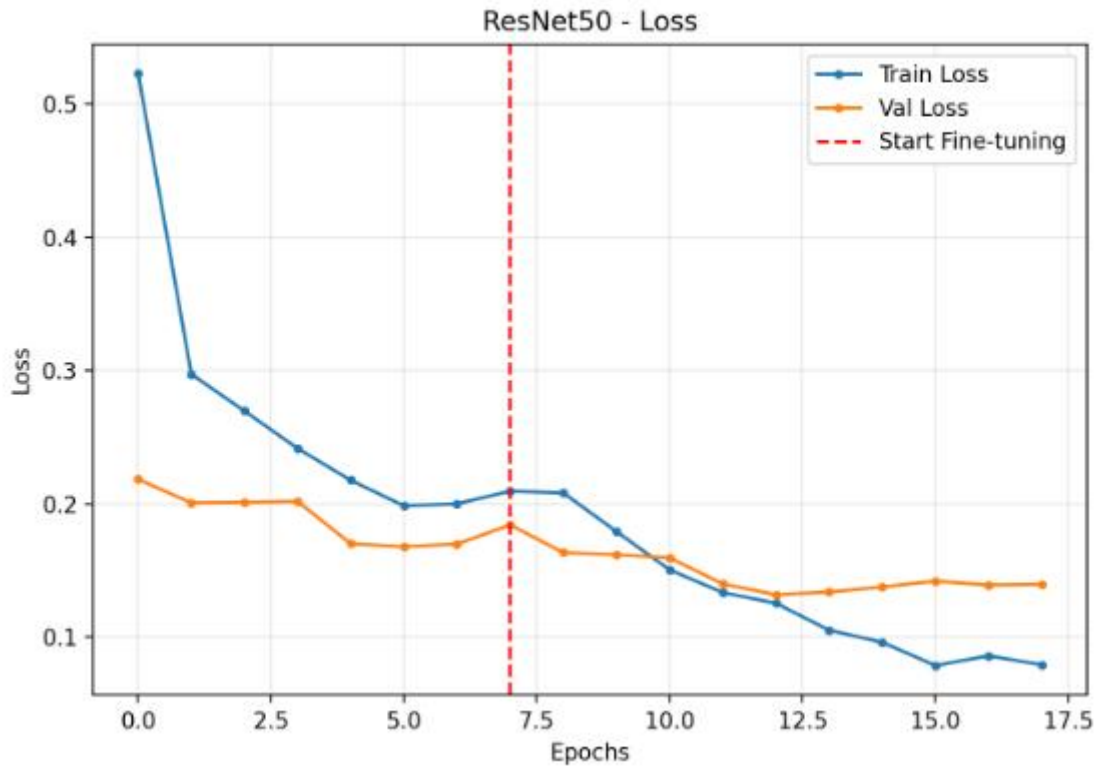
Salmonella:

- Precision: đạt 0,94. Cao, đảm bảo độ tin cậy cho kết quả dự đoán.
- Recall: đạt 0,97. Rất cao, cho thấy mô hình chỉ bỏ sót 3% mẫu Salmonella thực tế.
- F1 – score: đạt 0,95. Rất cao và ổn định. Kết quả dự đoán độ tin cậy cao.

Nhận xét: Chỉ số F1-score của mô hình dao động từ 0.93 đến 0.97, thể hiện sự cân bằng lý tưởng giữa độ chính xác (Precision) và độ nhạy (Recall). Đặc biệt, lớp Coccidiosis đạt giá trị F1-score cao nhất (0.97), cho thấy khả năng nhận diện đặc trưng ưu việt của EfficientNetB0 đối với chủng bệnh này. Lớp New Castle Disease dù có lượng mẫu hạn chế nhưng vẫn đạt F1-score mức 0.95, chứng minh rằng mô hình không bị hiện tượng thiên kiến dữ liệu và có khả năng phân loại ổn định trên tất cả các lớp bệnh lý khác nhau. Kết quả này khẳng định mô hình hoàn toàn đủ tin cậy để ứng dụng trong thực tế chẩn đoán thú y.

2. Resnet50:





- Nhận xét chi tiết – Accuracy

Giai đoạn Feature Extraction (trước vạch đỏ)

Train Accuracy: Tăng nhanh từ ~ 0.82 -- ~ 0.93

Cho thấy backbone ResNet50 trích xuất đặc trưng rất hiệu quả

Validation Accuracy: Luôn cao hơn Train Accuracy. Dao động quanh ~ 0.92 -- ~ 0.95

Nhận xét: Mô hình chưa bị overfitting

Giai đoạn Fine-tuning (sau vạch đỏ)

Train Accuracy: Tiếp tục tăng đều đạt ~ 0.97 – 0.98

Validation Accuracy: Tăng chậm hơn. Ổn định quanh ~ 0.96

Nhận xét:

Fine-tuning giúp mô hình học sâu hơn đặc trưng của dataset

Khoảng cách Train – Val không lớn, không có dấu hiệu overfitting nghiêm trọng

- Nhận xét chi tiết – Loss

Giai đoạn Feature Extraction

Train Loss: Giảm mạnh từ $\sim 0.52 \rightarrow \sim 0.20$

Validation Loss: Giảm từ $\sim 0.22 \rightarrow \sim 0.17$

Nhận xét: Loss giảm nhanh và ổn định. Mô hình học tốt ngay từ các epoch đầu nhờ pretrained weights

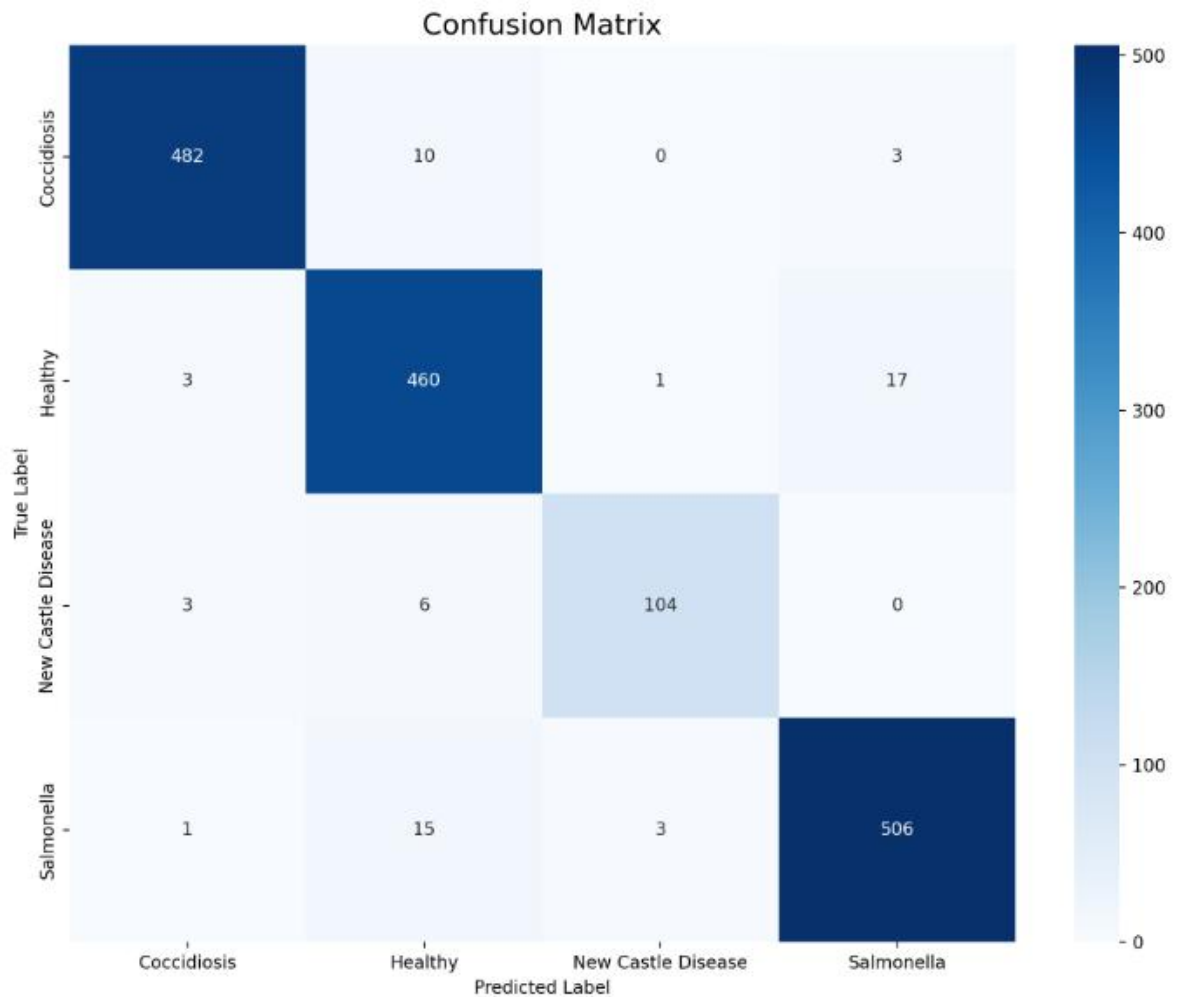
Giai đoạn Fine-tuning

Train Loss: Giảm tiếp, xuống ~ 0.08

Validation Loss: Giảm chậm hơn, dao động nhẹ quanh $\sim 0.13-0.14$

Nhận xét: Loss không tăng vọt, không bị overfitting. Validation loss ổn định, mô hình tổng quát hóa tốt

==> Biểu đồ Accuracy và Loss cho thấy mô hình ResNet50 hội tụ tốt qua hai giai đoạn Feature Extraction và Fine-tuning. Trong giai đoạn Feature Extraction, mô hình nhanh chóng đạt độ chính xác cao nhờ sử dụng trọng số pretrained từ ImageNet. Khi chuyển sang Fine-tuning, Train Accuracy tiếp tục tăng trong khi Validation Accuracy ổn định, đồng thời Loss của cả hai tập đều giảm. Điều này cho thấy mô hình học thêm được các đặc trưng đặc thù của bài toán mà không xảy ra hiện tượng overfitting đáng kể.



Độ chính xác cao trên tất cả các lớp

Không có lớp nào bị “sập” (recall quá thấp)

Học tốt cả lớp ít mẫu (New Castle Disease)

Healthy đôi khi bị nhầm sang Salmonella

==> Confusion Matrix cho thấy mô hình phân loại rất tốt với số lượng dự đoán đúng cao trên tất cả các lớp. Các giá trị tập trung chủ yếu trên đường chéo chính, cho thấy mô hình có độ chính xác cao. Một số nhầm lẫn nhỏ xảy ra giữa các lớp Healthy và Salmonella, có thể do sự tương đồng về đặc trưng hình ảnh trong một số trường hợp. Tuy nhiên, tổng thể mô hình vẫn đạt hiệu năng tốt và có khả năng tổng quát hóa cao.

	precision	recall	f1-score	support
Coccidiosis	0.99	0.97	0.98	495
Healthy	0.94	0.96	0.95	481
New Castle Disease	0.96	0.92	0.94	113
Salmonella	0.96	0.96	0.96	525
accuracy			0.96	1614
macro avg	0.96	0.95	0.96	1614
weighted avg	0.96	0.96	0.96	1614

- Precision (Độ chính xác dự đoán dương)

Trong số các mẫu mà mô hình dự đoán là lớp X, có bao nhiêu % là đúng thật

Ví dụ:

Precision Salmonella = 0.96

Trong 100 ảnh đoán Salmonella, 96 ảnh là đúng

- Recall (Độ bao phủ)

Trong số các mẫu thật sự thuộc lớp X, mô hình phát hiện được bao nhiêu %

Ví dụ:

Recall Coccidiosis = 0.97

Phát hiện được 97% số ca Coccidiosis thật

- F1-score: Trung bình điều hòa giữa Precision và Recall

- Support: Số lượng mẫu thật của từng lớp trong tập đánh giá

- Accuracy (Độ chính xác tổng thể): Tỷ lệ dự đoán đúng trên toàn bộ tập dữ liệu

- Macro Average: Trung bình các chỉ số của các lớp (coi các lớp ngang nhau)
- Weighted Average: Trung bình có xét số lượng mẫu (support)

✓ Coccidiosis

Precision: 0.99

Recall: 0.97

F1-score: 0.98

Nhận diện rất tốt, gần như không nhầm lẫn

Mô hình học được đặc trưng rõ ràng của bệnh này

✓ Healthy

Precision: 0.94

Recall: 0.96

F1-score: 0.95

Có một số trường hợp Healthy bị nhầm sang bệnh

Chấp nhận được và an toàn trong y tế (thà báo nhầm còn hơn bỏ sót)

✓ New Castle Disease

Precision: 0.96

Recall: 0.92

F1-score: 0.94

Support chỉ 113 ảnh

Dù số mẫu ít, mô hình vẫn đạt kết quả tốt

Cho thấy khả năng tổng quát hóa khá mạnh

✓ Salmonella

Precision: 0.96

Recall: 0.96

F1-score: 0.96

Cân bằng rất tốt giữa không báo nhầm và không bỏ sót

Phù hợp với bài toán chẩn đoán bệnh nguy hiểm

Accuracy = 96% cho thấy mô hình hoạt động hiệu quả trên toàn bộ tập dữ liệu.

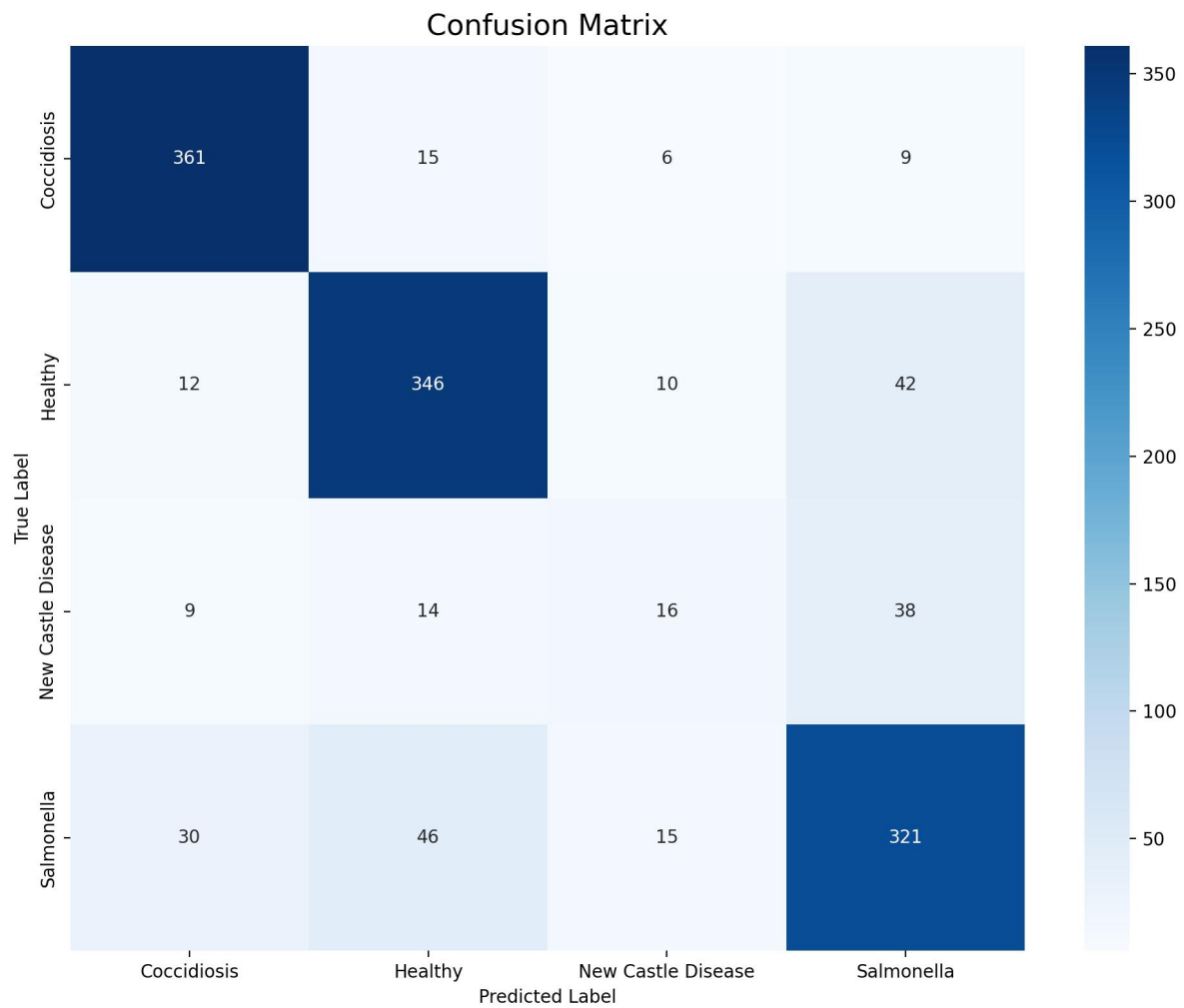
Macro avg ~ Weighted avg ~ 0.96: mô hình không thiên lệch mạnh về lớp nào.

Các lớp bệnh đều có F1-score ≥ 0.94 , kể cả lớp có ít dữ liệu.

Không xuất hiện lớp nào có recall thấp nghiêm trọng, ít bỏ sót bệnh.

==> Kết quả đánh giá cho thấy mô hình đạt độ chính xác tổng thể 96%. Các chỉ số precision, recall và F1-score đều cao và cân bằng giữa các lớp, với F1-score dao động từ 0.94 đến 0.98. Giá trị macro average và weighted average gần tương đương nhau cho thấy mô hình không bị ảnh hưởng nhiều bởi sự mất cân bằng dữ liệu và có khả năng tổng quát hóa tốt.

3. CNN



Mô hình CNN cơ bản đạt kết quả tốt với Coccidiosis và Healthy, tuy nhiên gặp khó khăn trong việc phân biệt Newcastle Disease do đặc trưng hình ảnh chồng lấn với Salmonella

	precision	recall	f1-score	support
Coccidiosis	0.88	0.92	0.90	391
Healthy	0.82	0.84	0.83	410
New Castle Disease	0.34	0.21	0.26	77
Salmonella	0.78	0.78	0.78	412
accuracy			0.81	1290
macro avg	0.71	0.69	0.69	1290
weighted avg	0.80	0.81	0.80	1290

==> Mô hình CNN đạt độ chính xác 81% trên tập validation. Các lớp Coccidiosis và Healthy được nhận diện tốt với F1-score lần lượt là 0.90 và 0.83. Tuy nhiên, mô hình gặp nhiều khó khăn trong việc phân biệt Newcastle Disease (F1-score = 0.26), nguyên nhân chủ yếu do số lượng dữ liệu hạn chế và đặc trưng hình ảnh chồng lấn với Salmonella. Điều này cho thấy CNN cơ bản chưa đủ khả năng học các đặc trưng phức tạp, cần sử dụng các mô hình sâu hơn như ResNet hoặc EfficientNet.

Chương 5: Kết quả bước đầu và nhận xét

1. Ưu và nhược điểm:

	Ưu điểm	Nhược điểm
CNN	- Kiến trúc đơn giản, dễ hiểu - Yêu cầu tài nguyên thấp - Làm mô hình baseline tốt	- Mô hình phải học toàn bộ đặc trưng từ đầu - Hiệu quả kém khi dữ liệu không đủ lớn. - Khó học được các đặc trưng phức tạp - Dễ bị overfitting
EfficientNetB0	- Tận dụng được các đặc trưng chung của ảnh - Khả năng tổng quát hóa tốt (phù hợp dataset vừa và nhỏ) - Cân bằng tốt độ chính xác, số lượng tham số, thời gian huấn luyện	- Khả năng học đặc trưng phức tạp chưa tối đa - Nếu không sử dụng đúng hàm preprocess_input thì hiệu quả giảm rõ rệt.
ResNet50	- Khả năng trích xuất đặc trưng rất mạnh - Phù hợp với dữ liệu ảnh y sinh và nông nghiệp	- Thời gian huấn luyện lâu, cần GPU mạnh - Nguy cơ overfitting nếu fine-tune sâu

2. Kết luận:

Qua quá trình đánh giá, mô hình CNN tự xây dựng có ưu điểm về tính đơn giản và yêu cầu tài nguyên thấp, tuy nhiên hạn chế lớn là khả năng trích xuất đặc trưng chưa tốt và dễ bị overfitting. EfficientNetB0 cho kết quả tốt hơn nhờ thiết kế tối ưu và sử dụng trọng số pretrained, giúp cải thiện khả năng tổng quát hóa. Trong khi đó, ResNet50 là mô hình cho hiệu suất cao nhất nhờ kiến trúc sâu và cơ chế residual learning, tuy nhiên đòi hỏi tài nguyên tính toán lớn hơn. Do đó, mỗi mô hình đều có ưu và nhược điểm riêng, phù hợp với những mục đích sử dụng khác nhau.

ResNet50 cho độ chính xác cao nhất, EfficientNetB0 cân bằng tốt giữa hiệu năng và tài nguyên, còn CNN phù hợp làm baseline.

Chương 6: WEB chẩn đoán

1. Giới thiệu chung:

Sau khi đã xây dựng và tối ưu hóa các mô hình học sâu, bước tiếp theo là triển khai chúng thành một ứng dụng thực tiễn nhằm hỗ trợ người chăn nuôi. Chương này tập trung vào việc xây dựng hệ thống chẩn đoán bệnh gia cầm dựa trên nền tảng Web thông qua thư viện Streamlit. Ứng dụng được thiết kế nhằm mục đích cung cấp một giao diện trực quan, cho phép so sánh song song kết quả dự đoán từ hai mô hình **ResNet50** và **EfficientNetB0**, từ đó giúp tăng tính khách quan và chính xác trong việc đưa ra kết luận về tình trạng sức khỏe của vật nuôi. Ứng dụng cho phép tải ảnh trực tiếp và nhận kết quả phân tích theo thời gian thực với các chỉ số thống kê minh bạch.

2. Source code:

- Thư viện sử dụng.

```
import os
import streamlit as st
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
from PIL import Image
import tensorflow as tf
```

Streamlit là một framework mã nguồn mở dành cho Python, giúp các nhà khoa học dữ liệu và lập trình viên dễ dàng xây dựng các ứng dụng web trực quan để phân tích dữ liệu và triển khai mô hình machine learning (ML). Ra mắt lần đầu vào năm 2019, Streamlit nhanh chóng trở thành lựa chọn phổ biến nhờ tính đơn giản, nhanh chóng và khả năng tạo giao diện người dùng mà không cần kinh nghiệm lập trình web.

- Import các hàm tiện xử lý cho các mô hình.

```
# Import các hàm tiện xử lý tương ứng cho từng mô hình
from tensorflow.keras.applications.resnet50 import preprocess_input as resnet_preprocess
from tensorflow.keras.applications.efficientnet import preprocess_input as eff_preprocess
```

- Cấu hình giao diện.

```
# CẤU HÌNH GIAO DIỆN
st.set_page_config(page_title="Hệ thống Chẩn đoán Bệnh Gà", layout="wide")
st.markdown("<h1 style='text-align: center;'>CHẨN ĐOÁN BỆNH GÀ</h1>", unsafe_allow_html=True)
st.markdown("---")
```

Thiết lập diện mạo và khung sườn cho trang web.

st.set_page_config(): Thiết lập tiêu đề trên tab trình duyệt.

layout="wide": Nội dung sẽ được trải dài ra toàn bộ chiều ngang của cửa sổ trình duyệt.

st.markdown("<h1 style='text-align: center;'>CHẨN ĐOÁN BỆNH GÀ</h1>", unsafe_allow_html=True): Tạo một tiêu đề nằm giữa trang web.

st.markdown("---"): Tạo một đường kẻ ngang ngăn cách tiêu đề và nội dung bên dưới

- Đường dẫn mô hình.

```
# CẤU HÌNH ĐƯỜNG DẪN
BASE_DIR = os.path.dirname(os.path.abspath(__file__))

# chỉnh: model nằm trong thư mục models/
MODEL_RESNET_PATH = os.path.join(BASE_DIR, "models", "resnet50_final2.keras")
MODEL_EFF_PATH = os.path.join(BASE_DIR, "models", "EfficientNetB0.keras")
```

BASE_DIR = os.path.dirname(os.path.abspath(__file__)): Giúp ứng dụng có thể hoạt động ở các nơi khác nhau. Câu lệnh sẽ tự động tìm đến thư mục chứa mô hình dự đoán của ứng dụng.

MODEL_RESNET_PATH: Trỏ vào file mô hình ResNet50 nằm trong thư mục con models.

MODEL_EFF_PATH: Trỏ vào file mô hình EfficientNetB0 cũng nằm trong thư mục models

SAMPLE_DIR: Trỏ vào thư mục data/samples, nơi chứa các ảnh gà có sẵn để người dùng chọn nhanh thay vì tải ảnh mới.

```
# XỬ LÝ LỖI CUSTOM OBJECTS
# Hàm giả lập để nạp được mô hình có sử dụng Focal Loss khi train
def focal_loss_fixed(y_true, y_pred):
    return y_pred
```

Do mô hình phân loại bệnh gà được huấn luyện với hàm mất mát tùy chỉnh (Focal Loss) việc nạp mô hình trực tiếp sẽ gây ra lỗi thiếu định nghĩa đối tượng (Custom Objects). Hệ thống đã giải quyết vấn đề này bằng cách định nghĩa một hàm giả lập `focal_loss_fixed`. Phương pháp này cho phép ứng dụng nạp thành công các trọng số đã được tối ưu hóa mà không cần cài đặt lại toàn bộ thư viện huấn luyện phức tạp.

- Nạp mô hình.

```
# NẠP MÔ HÌNH
@st.cache_resource
def load_models():
    custom_dict = {"focal_loss_fixed": focal_loss_fixed}
    # compile=False giúp nạp mô hình nhanh hơn và bỏ qua các lỗi hàm loss khi dự đoán
    res_model = tf.keras.models.load_model(MODEL_RESNET_PATH, custom_objects=custom_dict, compile=False)
    eff_model = tf.keras.models.load_model(MODEL_EFF_PATH, custom_objects=custom_dict, compile=False)
    return res_model, eff_model
```

Nhằm tối ưu hóa trải nghiệm người dùng và giảm thiểu tài nguyên tính toán, ứng dụng thực hiện cơ chế nạp mô hình thông qua decorator `@st.cache_resource`. Kỹ thuật này cho phép lưu trữ mô hình trong bộ nhớ đệm (Cache), tránh việc nạp lại dữ liệu nặng từ ổ cứng trong mỗi phiên làm việc.

```
try:
    model_res, model_eff = load_models()
except Exception as e:
    st.error(f"Lỗi nạp mô hình: {e}")
    st.info("Kiểm tra lại tên file mô hình trong thư mục đã khớp với code chưa.")
    st.stop() #dừng app để tránh gọi predict khi model chưa load
```

Chương trình sẽ thử thực hiện việc nạp mô hình thông qua hàm `load_models()`. Nếu mọi thứ bình thường (file có sẵn, đúng định dạng), chương trình sẽ chạy tiếp.

`except Exception as e`: Nếu trong quá trình nạp mô hình mà xuất hiện lỗi thì chương trình sẽ hiển thị các thông báo lỗi và dừng lại quá trình nạp.

```
# SIDEBAR
st.sidebar.header("Dữ liệu đầu vào")
uploaded = st.sidebar.file_uploader("Upload ảnh gà cần chẩn đoán", type=["jpg", "jpeg", "png"])

# Tự động quét ảnh trong thư mục samples nếu có
sample_paths = []
if os.path.exists(SAMPLE_DIR):
    sample_paths = [os.path.join(SAMPLE_DIR, f) for f in os.listdir(SAMPLE_DIR)
                    if f.lower().endswith((".jpg", ".jpeg", ".png"))]

sample_choice = st.sidebar.selectbox("Hoặc chọn ảnh mẫu có sẵn", ["(không)"] + sample_paths)
```

Người dùng có thể linh hoạt chọn hai phương thức up dữ liệu:

- Truy xuất dữ liệu trực tiếp từ thiết bị cá nhân thông qua giao thức tải tệp.

- Lựa chọn các mẫu thử nghiệm có sẵn từ cơ sở dữ liệu hệ thống.

Sau khi dữ liệu được xác nhận thành công, hệ thống sẽ kích hoạt luồng suy luận song song trên cả hai kiến trúc ResNet50 và EfficientNetB0 để đưa ra kết quả chẩn đoán.

- Logic dự đoán.

```
# LOGIC DỰ ĐOÁN
def predict_model(img_pil, model_type):
    img_resized = img_pil.resize(IMG_SIZE)
    img_array = np.array(img_resized, dtype=np.float32)
    img_array = np.expand_dims(img_array, axis=0)

    if model_type == "resnet":
        x = resnet_preprocess(img_array)
        probs = model_res.predict(x, verbose=0)[0]
    else:
        x = eff_preprocess(img_array)
        probs = model_eff.predict(x, verbose=0)[0]

    idx = int(np.argmax(probs))
    return idx, probs
```

img_pil.resize(IMG_SIZE): chuẩn hóa kích thước ảnh về 224x 224.

np.array(img_resized, dtype=np.float32): chuyển ảnh thành một ma trận các con số từ 0 đến 255. Kiểu dữ liệu float32 (số thực) giúp các phép tính nhân ma trận sau này của AI chính xác hơn.

np.expand_dims(img_array, axis=0): biến 1 tấm ảnh thành 1 "gói" chứa 1 tấm ảnh, chuyển cấu trúc từ (224, 224, 3) sang (1, 224, 224, 3).

```
if model_type == "resnet":
    x = resnet_preprocess(img_array)
    probs = model_res.predict(x, verbose=0)[0]
else:
    x = eff_preprocess(img_array)
    probs = model_eff.predict(x, verbose=0)[0]

idx = int(np.argmax(probs))
return idx, probs
```

Tiền xử lý theo mô hình:

- Nếu là **ResNet**: Dùng `resnet_preprocess` để trừ giá trị trung bình màu và đổi hệ màu sang BGR.

- Nếu là **EfficientNet**: Dùng `eff_preprocess` để đưa các giá trị pixel về khoảng $[0, 1]$.

predict(x, verbose=0): Thực hiện quá trình suy luận. `verbose=0` giúp ẩn đi các dòng thông báo tiến trình chạy trong màn hình console.

probs: Kết quả trả về từ `model.predict` là một danh sách 4 con số (tương ứng với 4 loại bệnh), tổng của chúng bằng 1.

np.argmax(probs): Tìm vị trí (chỉ số) của con số lớn nhất trong danh sách.

Ví dụ: Nếu kết quả là $[0.1, 0.8, 0.05, 0.05]$, con số lớn nhất nằm ở vị trí thứ 2 (chỉ số 1).

return idx, probs: Trả về cả tên bệnh (dưới dạng số thứ tự) và danh sách xác suất chi tiết để vẽ biểu đồ.

Hàm **predict_model** thực thi quy trình biến đổi dữ liệu (Data Transformation Pipeline) gồm các bước: chuẩn hóa kích thước, chuyển đổi sang không gian số thực và đồng nhất hóa cấu trúc mảng đầu vào theo chuẩn Batch-processing. Đặc biệt, hàm được thiết kế linh hoạt để áp dụng các kỹ thuật tiền xử lý (Preprocessing) riêng biệt cho từng kiến trúc mạng khác nhau, giúp tối ưu hóa độ chính xác cho cả ResNet50 và EfficientNetB0. Kết quả đầu ra bao gồm chỉ số nhãn bệnh có xác suất cao nhất và mảng phân phối xác suất trên toàn bộ các lớp đối tượng.

- Lấy ảnh từ upload hoặc ảnh mẫu:

```
# HIỂN THỊ KẾT QUẢ
img = None
✓ if uploaded is not None:
    |     img = Image.open(uploaded).convert("RGB")
✓ elif sample_choice != "(không)":
    |     img = Image.open(sample_choice).convert("RGB")
```


- Nếu user upload: ưu tiên dùng upload.
 - Nếu không upload mà có chọn ảnh mẫu: dùng ảnh mẫu.
 - `convert("RGB")`:
 - chuẩn hoá ảnh về 3 kênh RGB
 - tránh lỗi ảnh PNG có alpha (RGBA) hoặc ảnh xám (L).
- Canh ảnh vào giữa màn hình bằng `columns`:

```
if img is not None:
    # Hiển thị ảnh gốc ở giữa
    # Cột trống ở hai bên sẽ ép cột giữa vào trung tâm màn hình
    left_co, cent_co, last_co = st.columns([1, 2, 1])

    with cent_co:
        st.subheader("Ảnh đang kiểm tra")
        st.image(img, use_container_width=True)

    # Chia 2 cột để so sánh 2 mô hình
    col1, col2 = st.columns(2)
```

- `st.columns([1,2,1])` tạo 3 cột:

cột trái rộng 1

cột giữa rộng 2

cột phải rộng 1

Nhờ vậy cột giữa nằm “giữa” màn hình.

- `with cent_co::` mọi lệnh `st.xxx` bên trong sẽ vẽ vào cột giữa.
- `st.subheader(...)`: tiêu đề nhỏ.
- `st.image(img, use_container_width=True)`:
hiển thị ảnh
tự co giãn theo bề rộng container.

- Hiện thị vòng xoay loading trong lúc chạy:

```
# Thực hiện dự đoán
with st.spinner('Đang tính toán...'):
    idx_res, probs_res = predict_model(img, "resnet")
    idx_eff, probs_eff = predict_model(img, "eff")
```

- Hiện thị kết quả ResNet50

```
# Cột 1: ResNet50
with col1:
    st.subheader("1. Kết quả ResNet50")
    st.metric(label="Chẩn đoán", value=LABELS[idx_res], delta=f"{probs_res[idx_res]*100:.2f}%")
```

with col1: vẽ nội dung vào cột trái.

st.metric(...) hiển thị dạng thẻ KPI:

label: tên chỉ số (Chẩn đoán)

value: giá trị chính (tên bệnh)

delta: dòng phụ (hiện % xác suất)

- Vẽ biểu đồ cột xác suất

```
fig1, ax1 = plt.subplots()
colors = ['gray'] * len(LABELS)
colors[idx_res] = 'skyblue'
ax1.bar(LABELS, probs_res, color=colors)
plt.xticks(rotation=45)
ax1.set_title("Xác suất từng lớp (ResNet50)")
st.pyplot(fig1)
```

plt.subplots() tạo “khung hình” fig và “trục vẽ” ax.

colors = ['gray'] * len(LABELS): tạo mảng màu mặc định xám cho mọi cột.

colors[idx_res] = 'skyblue': tô nổi cột có xác suất cao nhất.

ax1.bar(LABELS, probs_res, color=colors) vẽ cột.

plt.xticks(rotation=45) xoay nhãn trục x cho dễ đọc.

st.pyplot(fig1) đưa hình matplotlib lên Streamlit.

- Kẻ đường phân cách & bảng so sánh

```
# Bảng so sánh chi tiết
st.divider()
st.subheader("Bảng so sánh chi tiết (%)")
```

- Tạo DataFrame

```
df_compare = pd.DataFrame({
    "Loại Bệnh / Sức khỏe": LABELS,
    "ResNet50 (%)": [f"{p*100:.2f}" for p in probs_res],
    "EfficientNetB0 (%)": [f"{p*100:.2f}" for p in probs_eff]
})
st.table(df_compare)
```

d.DataFrame({ ... }) tạo bảng từ dict cột.

Cột "Loại Bệnh / Sức khỏe": danh sách nhãn.

Cột "ResNet50 (%)":

list comprehension: [f"{p*100:.2f}" for p in probs_res]

duyệt từng p trong mảng xác suất

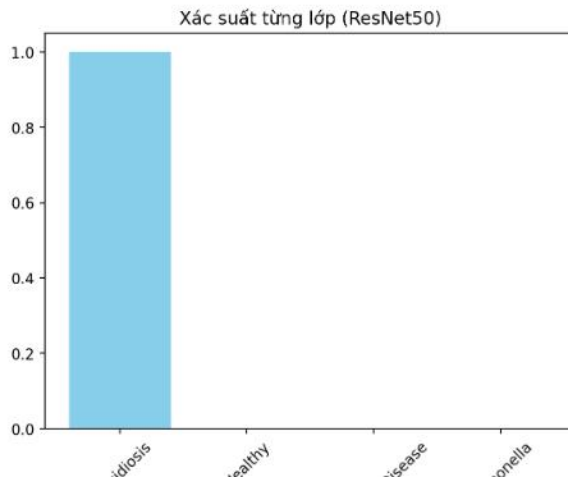
đổi sang phần trăm và format 2 chữ số.

1. Kết quả ResNet50

Chẩn đoán

Coccidiosis

↑ 100.00%

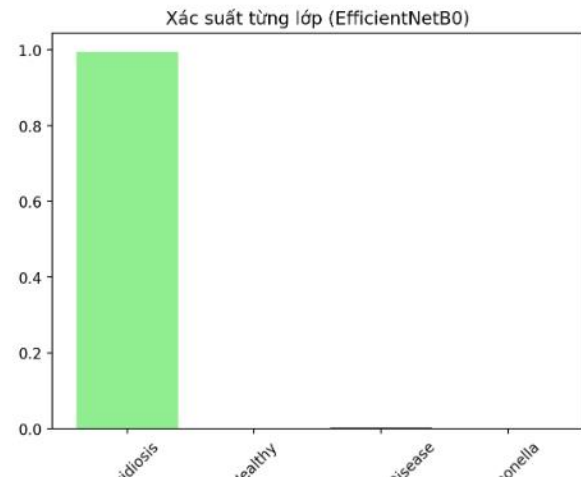


2. Kết quả EfficientNetB0

Chẩn đoán

Coccidiosis

↑ 99.44%



Bảng so sánh chi tiết (%)

	Loại Bệnh / Sức khỏe	ResNet50 (%)	EfficientNetB0 (%)
0	Coccidiosis	100.00	99.44
1	Healthy	0.00	0.09
2	New Castle Disease	0.00	0.41
3	Salmonella	0.00	0.06

Chương 7: Các tài liệu tham khảo

[1] Allandclive, Chicken Disease Dataset, Kaggle.

<https://www.kaggle.com/datasets/allandclive/chicken-disease-1>

[2] Y. LeCun, Y. Bengio, and G. Hinton, “Deep learning,” Nature, vol. 521, no. 7553, pp. 436–444, 2015.

[3] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” NeurIPS, 2012.

Bảng phân công công việc nhóm 3

Tên thành viên	Nhiệm vụ
Nguyễn Quốc Bảo	Mô hình EfficientNetB0, chạy demo và Word liên quan
Cao Thị Ngọc Luyến	Mô hình ResNet50, chạy demo và Word liên quan
Lục Duy Quang	Mô hình CNN và Word liên quan